

循序渐进，学习开发 xv6-riscv

第 18 章 进程的创建、执行和回收

汪辰



目录

01

进程的操作模型

02

退出进程 (exit)

03

执行程序 (exec)

04

创建进程 (fork)

05

回收进程 (wait)

06

终止进程 (kill)



01

进程的操作模型



进程操作模型

```
int pid, wpid, wstatus;

pid = fork(); // 父进程创建子进程
if (pid < 0) {
    // 创建进程失败
    return;
}

if (pid == 0) {
    // 子进程路径
    exec("xxxx", argv); // 执行程序 xxxx
    // 如果执行程序成功, exec 不会返回
    exit(1);
}

// 父进程路径
wpid = wait(&wstatus); // 父进程等待任一个子进程结束
if (wpid == pid) {
    // 子进程退出, wpid 是退出的子进程的 pid
    // wstatus 是子进程退出的状态值
} else {
    // wait 失败返回
}
```

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc with dots at its ends is positioned around the top and right of the circles. In the top right corner, there is a solid blue rectangle.

02

退出进程 (exit)

进程操作模型 - exit

```
int pid, wpid, wstatus;

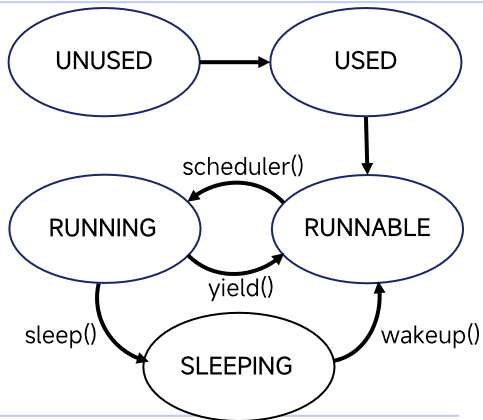
pid = fork(); // 父进程创建子进程
if (pid < 0) {
    // 创建进程失败
    return;
}

if (pid == 0) {
    // 子进程路径
    exec("xxxx", argv); // 执行程序 xxxx
    // 如果执行程序成功, exec 不会返回
    exit(1); 
}

// 父进程路径
wpid = wait(&wstatus); // 父进程等待任一个子进程结束
if (wpid == pid) {
    // 子进程退出, wpid 是退出的子进程的 pid
    // wstatus 是子进程退出的状态值
} else {
    // wait 失败返回
}
```

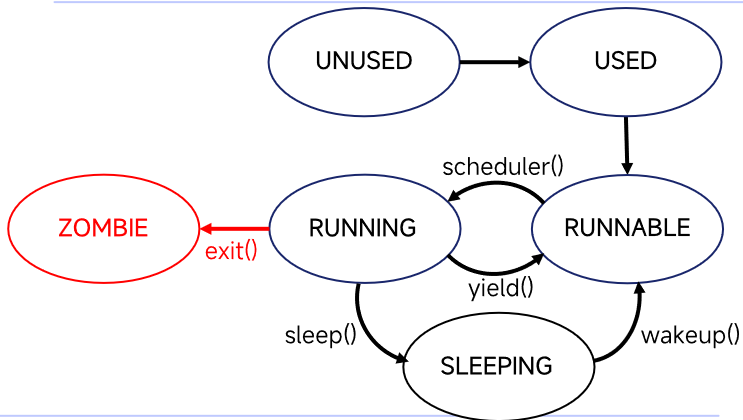
退出进程 (exit)

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```



退出进程 (exit)

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```



退出进程 (exit)

```
exit(2)                                System Calls Manual                                exit(2)

NAME
    _exit, _Exit - terminate the calling process

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <unistd.h>

    [[noreturn]] void _exit(int status);
```

man 2 exit

退出进程 (exit)

```
__exit(2)                                System Calls Manual                                __exit(2)

NAME
    _exit, _Exit - terminate process

LIBRARY
    Standard C library (libc)

SYNOPSIS
    #include <unistd.h>

    [[noreturn]] void _exit(int status);
```

user/user.h

int exit(int status) __attribute__((noreturn));

退出进程 (exit)

```
exit(2)                                System Calls Manual                                exit(2)

NAME
    _exit, _Exit - terminate process

LIBRARY
    Standard C library (libc)

SYNOPSIS
    #include <unistd.h>

    [[noreturn]] void _exit(int status);
```

user/user.h

int exit(int status) __attribute__((noreturn));

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index e0d73cf..d48d55b 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -46,6 +46,7 @@ argaddr(int n, uint64 *ip)
 }

// Prototypes for the functions that handle system calls.
+extern uint64 sys_exit(void);
extern uint64 sys_read(void);
extern uint64 sys_getpid(void);
extern uint64 sys_pause(void);
@@ -55,6 +56,7 @@ extern uint64 sys_write(void);
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
+ [SYS_exit]    sys_exit,
 [SYS_read]    sys_read,
 [SYS_getpid]  sys_getpid,
 [SYS_pause]   sys_pause,
```

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index f2c44ca..5246f73 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
@@ -6,6 +6,15 @@
 #include "spinlock.h"
 #include "proc.h"

+uint64
+sys_exit(void)
+{
+    int n;
+    argint(0, &n);
+    kexit(n);
+    return 0; // not reached
+}
```

退出进程 (exit)

```
diff --git a/kernel/proc.h b/kernel/proc.h
index e3b51b1..08ad307 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -88,6 +88,7 @@ struct proc {
    // p->lock must be held when using these:
    enum procstate state;           // Process state
    void *chan;                     // If non-zero, sleeping on chan
+   int xstate;                     // Exit status to be returned to parent's wait
    int pid;                        // Process ID
```

```
void kexit(int status)
{
    struct proc *p = myproc();

    acquire(&p->lock);

    p->xstate = status;
    p->state = ZOMBIE;

    // Jump into the scheduler, never to return.
    sched();
    panic("zombie exit");
}
```

退出进程 (exit)

user/init.c

```
void main(void)
{
    static char buf[100];
    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        char *cmd = buf;
        // .....
        runcmd(cmd);
    }
}

void runcmd(char *cmd)
{
    if(strcmp(cmd, "exit") == 0) {
        printf("Exiting shell...\n");
        exit(0);
    } else
        printf("Running command: \'%s\'', unsupported!\n", cmd);
}
```

退出进程 (exit)

user/init.c

```
void main(void)
{
    static char buf[100];
    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0)
    {
        char *cmd = buf;
        // .....
        runcmd(cmd);
    }
}

void runcmd(char *cmd)
{
    if(strcmp(cmd, "exit") == 0) {
        printf("Exiting shell...\n");
        exit(0);
    } else
        printf("Running command: '%s', unsupported!\n", cmd);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel
kernel/kernel -m 128M -smp 3 -nographic
```

xv6 kernel is booting

hart 1 starting

hart 2 starting

\$ exit

Exiting shell...

退出进程 (exit)

user/init.c

```
void main(void)
{
    static char buf[100];
    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0)
        char *cmd = buf;
        // .....
        runcmd(cmd);
}
```

```
void runcmd(char *cmd)
{
    if(strcmp(cmd, "exit") == 0) {
        printf("Exiting shell...\n");
        exit(0);
    } else
        printf("Running command: '%s', unsupported!\n", cmd);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel
kernel/kernel -m 128M -smp 3 -nographic
```

xv6 kernel is booting

hart 1 starting
hart 2 starting
\$ exit

Exiting shell...

Ctrl + P

1 zombie init

退出进程 (exit)

user/init.c

```
void main(void)
{
    static char buf[100];
    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0)
    {
        char *cmd = buf;
        // .....
        runcmd(cmd);
    }
}

void runcmd(char *cmd)
{
    if(strcmp(cmd, "exit") == 0) {
        printf("Exiting shell...\n");
        exit(0);
    } else
        printf("Running command: '%s', unsupported!\n", cmd);
}
```

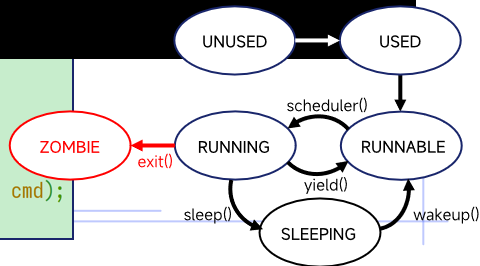
```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel
kernel/kernel -m 128M -smp 3 -nographic
```

xv6 kernel is booting

hart 1 starting
hart 2 starting
\$ exit
Exiting shell...

Ctrl + P

1 zombie init





03

执行程序 (exec)

进程操作模型 - exec

```
int pid, wpid, wstatus;

pid = fork(); // 父进程创建子进程
if (pid < 0) {
    // 创建进程失败
    return;
}

if (pid == 0) {
    // 子进程路径
    exec("xxxx", argv); // 执行程序 xxxx
    // 如果执行程序成功, exec 不会返回
    exit(1);
}

// 父进程路径
wpid = wait(&wstatus); // 父进程等待任一个子进程结束
if (wpid == pid) {
    // 子进程退出, wpid 是退出的子进程的 pid
    // wstatus 是子进程退出的状态值
} else {
    // wait 失败返回
}
```

执行程序 (exec)

- 系统调用 exec 的实现
- 核心函数 kexec 的实现
- init 程序的加载

执行程序 (exec)

- **系统调用 exec 的实现**
- 核心函数 kexec 的实现
- init 程序的加载

系统调用 exec

```
exec(3)                                Library Functions Manual                                exec(3)

NAME
    execl, execlp, execl, execv, execvp, execvpe - execute a file

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <unistd.h>

    extern char **environ;

    int execl(const char *pathname, const char *arg, ...
                /*, (char *) NULL */);
    int execlp(const char *file, const char *arg, ...
                /*, (char *) NULL */);
    int execl(const char *pathname, const char *arg, ...
                /*, (char *) NULL, char *const envp[] */);
    int execv(const char *pathname, char *const argv[]);
    int execvp(const char *file, char *const argv[]);
    int execvpe(const char *file, char *const argv[], char *const envp[]);
```

man 3 exec

系统调用 exec

```
exec(3)                                Library Functions Manual                                exec(3)

NAME
    execl, execlp, execl, execv, execvp, execvpe - execute a file

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <unistd.h>

    extern char **environ;

    int execl(const char *pathname, const char *arg0, ...,
              /*, (char *) NULL, char **argv */);
    int execlp(const char *file, const char *arg0, ...,
              /*, (char *) NULL, char **argv */);
    int execlx(const char *pathname, const char *arg0, ...,
              /*, (char *) NULL, char **argv */);
    int execv(const char *pathname, char *const argv[]);
    int execvp(const char *file, char *const argv[]);
    int execvpe(const char *file, char *const argv[], char *const envp[]);
```

man 3 exec

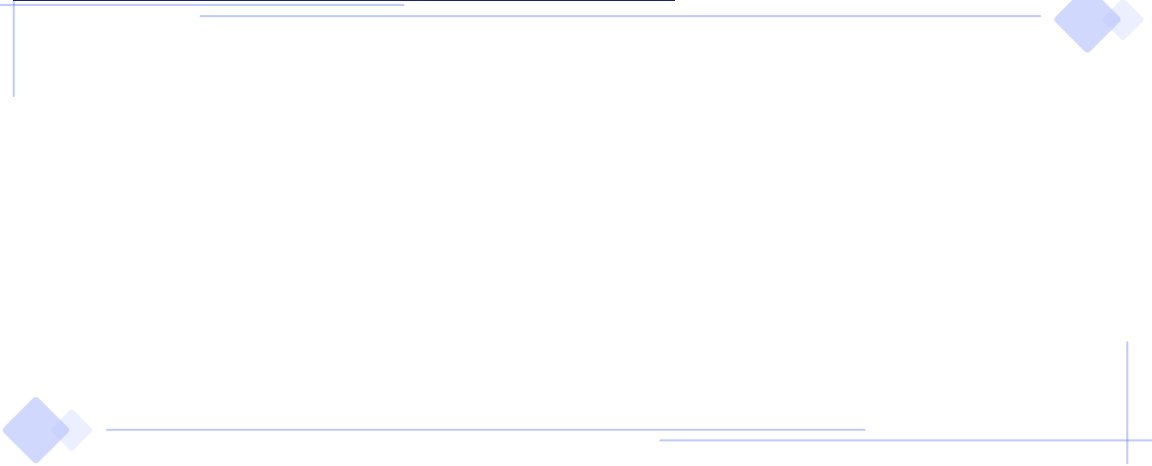
user/user.h

int exec(const char *pathname, char **argv);

系统调用 **exec**

user/user.h

```
int exec(const char *pathname , char **argv);
```



系统调用 **exec**

user/user.h

```
int exec(const char *pathname , char **argv);
```



系统调用 exec

user/user.h

```
int exec(const char *pathname, char **argv);
```

argv

argv[0]

"init"

argv[1]

"hello"

argv[2]

"world"

argv[3]

NULL

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index d48d55b..3449c03 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
.....
// Prototypes for the functions that handle system calls.
extern uint64 sys_exit(void);
extern uint64 sys_read(void);
+extern uint64 sys_exec(void);
extern uint64 sys_getpid(void);
extern uint64 sys_pause(void);
extern uint64 sys_uptime(void);
@@ -58,6 +93,7 @@ extern uint64 sys_write(void);
static uint64 (*syscalls[])(void) = {
[SYS_exit]    sys_exit,
[SYS_read]    sys_read,
+[SYS_exec]    sys_exec,
[SYS_getpid]  sys_getpid,
[SYS_pause]   sys_pause,
[SYS_uptime]  sys_uptime,
```

```
diff --git a/kernel/sysfile.c b/kernel/sysfile.c
index c11e886..596cb0f 100644
--- a/kernel/sysfile.c
+++ b/kernel/sysfile.c
@@ -39,3 +39,46 @@ sys_write(void)
    return consolewrite(1, p, n);
}
+
+uint64
+sys_exec(void)
+{
+    char path[MAXPATH], *argv[MAXARG];
+    int i;
+    uint64 uargv, uarg;
+
+    argaddr(1, &uargv);
+    if(argstr(0, path, MAXPATH) < 0) {
+        return -1;
+    }
+}
.....
```

系统调用 exec

user/user.h

```
int exec(const char *pathname, char **argv);
```

argv

argv[0]

"init"

argv[1]

"hello"

argv[2]

"world"

argv[3]

NULL

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index d48d55b..3449c03 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
.....
// Prototypes for the functions that handle system calls.
extern uint64 sys_exit(void);
extern uint64 sys_read(void);
+extern uint64 sys_exec(void);
extern uint64 sys_getpid(void);
extern uint64 sys_pause(void);
extern uint64 sys_uptime(void);
@@ -58,6 +93,7 @@ extern uint64 sys_write(void);
static uint64 (*syscalls[])(void) = {
[SYS_exit] sys_exit,
[SYS_read] sys_read,
+[SYS_exec] sys_exec,
[SYS_getpid] sys_getpid,
[SYS_pause] sys_pause,
[SYS_uptime] sys_uptime,
```

```
diff --git a/kernel/sysfile.c b/kernel/sysfile.c
index c11e886..596cb0f 100644
--- a/kernel/sysfile.c
+++ b/kernel/sysfile.c
@@ -39,3 +39,46 @@ sys_write(void)
    return consolewrite(1, p, n);
}
+uint64
+sys_exec(void)
+{
+    char path[MAXPATH], *argv[MAXARG];
+    int i;
+    uint64 uargv, uarg;
+    argaddr(1, &uargv);
+    if(argstr(0, path, MAXPATH) < 0) {
+        return -1;
+    }
+    memset(argv, 0, sizeof(argv));
+    for(i=0;; i++){
+        if(i >= NELEM(argv)){
+            goto bad;
+        }
+        if(fetchaddr(uargv+sizeof(uint64)*i, (uint64*)&uarg) < 0){
+            goto bad;
+        }
+        if(uarg == 0){
+            argv[i] = 0;
+            break;
+        }
+        argv[i] = kalloc();
+        if(argv[i] == 0)
+            goto bad;
+        if(fetchstr(uarg, argv[i], PGSIZE) < 0)
+            goto bad;
+    }
+    int ret = kexec(path, argv);
+    for(i = 0; i < NELEM(argv) && argv[i] != 0; i++)
+        kfree(argv[i]);
+    return ret;
+bad:
+    for(i = 0; i < NELEM(argv) && argv[i] != 0; i++)
+        kfree(argv[i]);
+    return -1;
}
```

```
uint64 sys_exec(void)
{
    char path[MAXPATH], *argv[MAXARG];
    int i;
    uint64 uargv, uarg;
    argaddr(1, &uargv);
    if(argstr(0, path, MAXPATH) < 0) {
        return -1;
    }
    memset(argv, 0, sizeof(argv));
    for(i=0;; i++){
        if(i >= NELEM(argv)){
            goto bad;
        }
        if(fetchaddr(uargv+sizeof(uint64)*i, (uint64*)&uarg) < 0){
            goto bad;
        }
        if(uarg == 0){
            argv[i] = 0;
            break;
        }
        argv[i] = kalloc();
        if(argv[i] == 0)
            goto bad;
        if(fetchstr(uarg, argv[i], PGSIZE) < 0)
            goto bad;
    }
    int ret = kexec(path, argv);
    for(i = 0; i < NELEM(argv) && argv[i] != 0; i++)
        kfree(argv[i]);
    return ret;
bad:
    for(i = 0; i < NELEM(argv) && argv[i] != 0; i++)
        kfree(argv[i]);
    return -1;
}
```

系统调用 exec

user/user.h

```
int exec(const char *pathname, char **argv);
```

argv

argv[0]

"init"

argv[1]

"hello"

argv[2]

"world"

argv[3]

NULL

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index d48d55b..3449c03 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
.....
// Prototypes for the functions that handle system calls.
extern uint64 sys_exit(void);
extern uint64 sys_read(void);
+extern uint64 sys_exec(void);
extern uint64 sys_getpid(void);
extern uint64 sys_pause(void);
extern uint64 sys_uptime(void);
@@ -58,6 +93,7 @@ extern uint64 sys_write(void);
static uint64 (*syscalls[])(void) = {
[SYS_exit] sys_exit,
[SYS_read] sys_read,
+[SYS_exec] sys_exec,
[SYS_getpid] sys_getpid,
[SYS_pause] sys_pause,
[SYS_uptime] sys_uptime,
```

```
diff --git a/kernel/sysfile.c b/kernel/sysfile.c
index c11e886..596cb0f 100644
--- a/kernel/sysfile.c
+++ b/kernel/sysfile.c
@@ -39,3 +39,46 @@ sys_write(void)
return consolewrite(1, p, n);
}
+
+uint64
+sys_exec(void)
+{
+ char path[MAXPATH], *argv[MAXARG];
+ int i;
+ uint64 uargv, uarg;
+
+ argaddr(1, &uargv);
+ if(argstr(0, path, MAXPATH) < 0) {
+ return -1;
+ }
+
+ .....
```

```
uint64 sys_exec(void)
{
char path[MAXPATH], *argv[MAXARG];
int i;
uint64 uargv, uarg;
argaddr(1, &uargv);
if(argstr(0, path, MAXPATH) < 0) {
return -1;
}
memset(argv, 0, sizeof(argv));
for(i=0;; i++){
if(i >= NELEM(argv)){
goto bad;
}
if(fetchaddr(uargv+sizeof(uint64)*i, (uint64*)&uarg) < 0){
goto bad;
}
if(uarg == 0){
argv[i] = 0;
break;
}
argv[i] = kalloc();
if(argv[i] == 0)
goto bad;
if(fetchstr(uarg, argv[i], PGSIZE) < 0)
goto bad;
}

int ret = kexec(path, argv);
for(i = 0; i < NELEM(argv) && argv[i] != 0; i++)
kfree(argv[i]);
return ret;
bad:
for(i = 0; i < NELEM(argv) && argv[i] != 0; i++)
kfree(argv[i]);
return -1;
}
```

系统调用 exec

user/user.h

```
int exec(const char *pathname, char **argv);
```

argv

argv[0]

"init"

argv[1]

"hello"

argv[2]

"world"

argv[3]

NULL

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index d48d55b..3449c03 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
.....
// Prototypes for the functions that handle system calls.
extern uint64 sys_exit(void);
extern uint64 sys_read(void);
+extern uint64 sys_exec(void);
extern uint64 sys_getpid(void);
extern uint64 sys_pause(void);
extern uint64 sys_uptime(void);
@@ -58,6 +93,7 @@ extern uint64 sys_write(void);
static uint64 (*syscalls[])(void) = {
[SYS_exit] sys_exit,
[SYS_read] sys_read,
+[SYS_exec] sys_exec,
[SYS_getpid] sys_getpid,
[SYS_pause] sys_pause,
[SYS_uptime] sys_uptime,
```

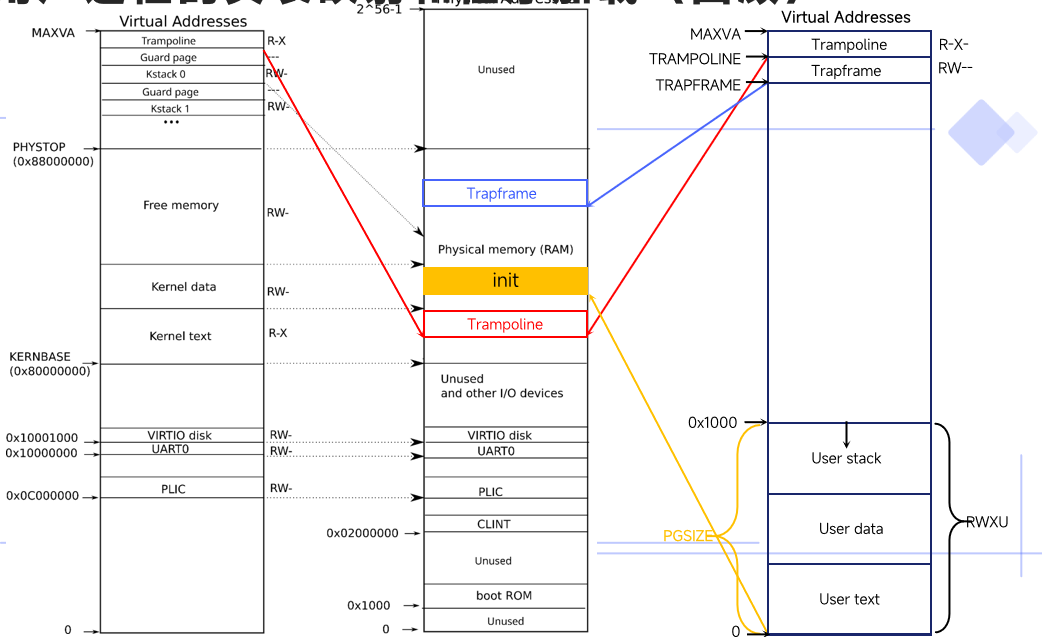
```
diff --git a/kernel/sysfile.c b/kernel/sysfile.c
index c11e886..596cb0f 100644
--- a/kernel/sysfile.c
+++ b/kernel/sysfile.c
@@ -39,3 +39,46 @@ sys_write(void)
    return consolewrite(1, p, n);
}
+uint64
+sys_exec(void)
+{
+    char path[MAXPATH], *argv[MAXARG];
+    int i;
+    uint64 uargv, uarg;
+
+    argaddr(1, &uargv);
+    if(argstr(0, path, MAXPATH) < 0) {
+        return -1;
+    }
+
+    memset(argv, 0, sizeof(argv));
+    for(i=0;; i++){
+        if(i >= NELEM(argv)){
+            goto bad;
+        }
+        if(fetchaddr(uargv+sizeof(uint64)*i, (uint64*)&uarg) < 0){
+            goto bad;
+        }
+        if(uarg == 0){
+            argv[i] = 0;
+            break;
+        }
+        argv[i] = kalloc();
+        if(argv[i] == 0)
+            goto bad;
+        if(fetchstr(uarg, argv[i], PGSIZE) < 0)
+            goto bad;
+    }
+
+    int ret = kexec(path, argv);
+    for(i=0; i < NELEM(argv) && argv[i] != 0; i++)
+        kfree(argv[i]);
+    return ret;
+bad:
+    for(i=0; i < NELEM(argv) && argv[i] != 0; i++)
+        kfree(argv[i]);
+    return -1;
+}
```

```
uint64 sys_exec(void)
{
    char path[MAXPATH], *argv[MAXARG];
    int i;
    uint64 uargv, uarg;
    argaddr(1, &uargv);
    if(argstr(0, path, MAXPATH) < 0) {
        return -1;
    }
    memset(argv, 0, sizeof(argv));
    for(i=0;; i++){
        if(i >= NELEM(argv)){
            goto bad;
        }
        if(fetchaddr(uargv+sizeof(uint64)*i, (uint64*)&uarg) < 0){
            goto bad;
        }
        if(uarg == 0){
            argv[i] = 0;
            break;
        }
        argv[i] = kalloc();
        if(argv[i] == 0)
            goto bad;
        if(fetchstr(uarg, argv[i], PGSIZE) < 0)
            goto bad;
    }
    int ret = kexec(path, argv);
    for(i=0; i < NELEM(argv) && argv[i] != 0; i++)
        kfree(argv[i]);
    return ret;
bad:
    for(i=0; i < NELEM(argv) && argv[i] != 0; i++)
        kfree(argv[i]);
    return -1;
}
```

执行程序 (exec)

- 系统调用 `exec` 的实现
- **核心函数 `kexec` 的实现**
- `init` 程序的加载

早期用户进程的页表映射和程序加载（回顾）



新的用户进程地址空间

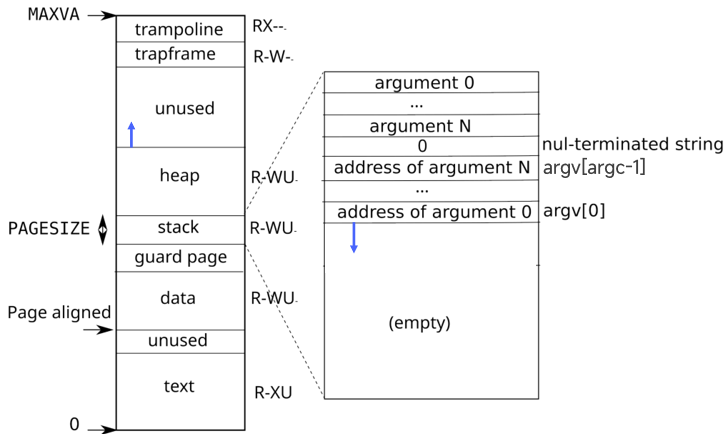


Figure 3.4: A process's user address space, with its initial stack.

新的用户程序加载

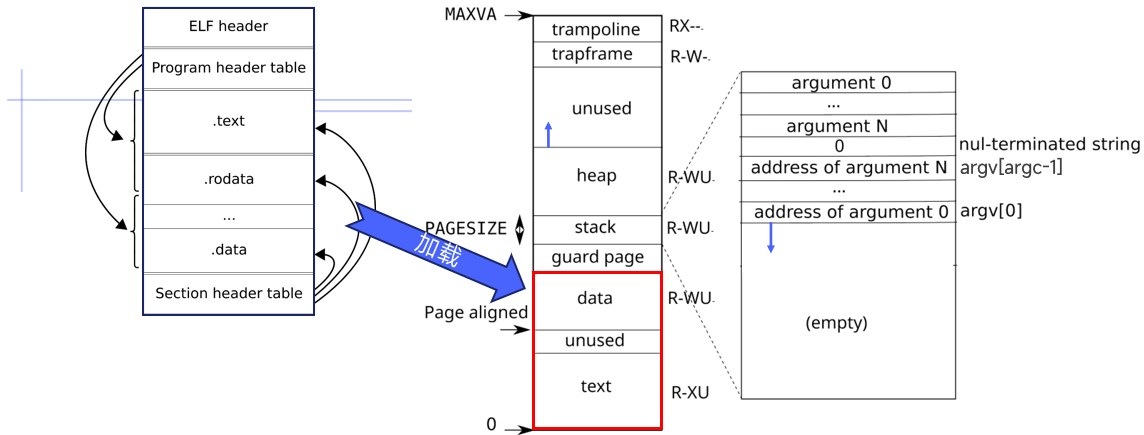


Figure 3.4: A process's user address space, with its initial stack.

新的用户程序加载

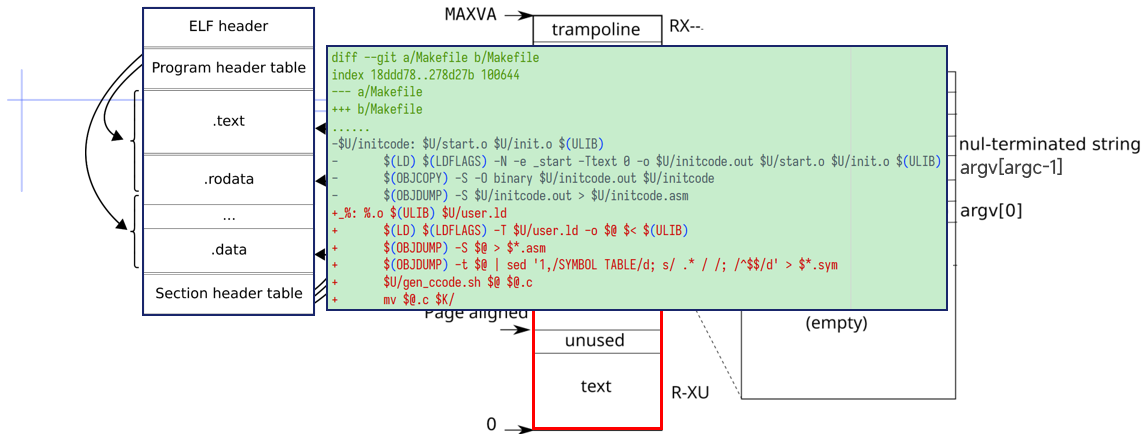


Figure 3.4: A process's user address space, with its initial stack.

完善的用户程序加载

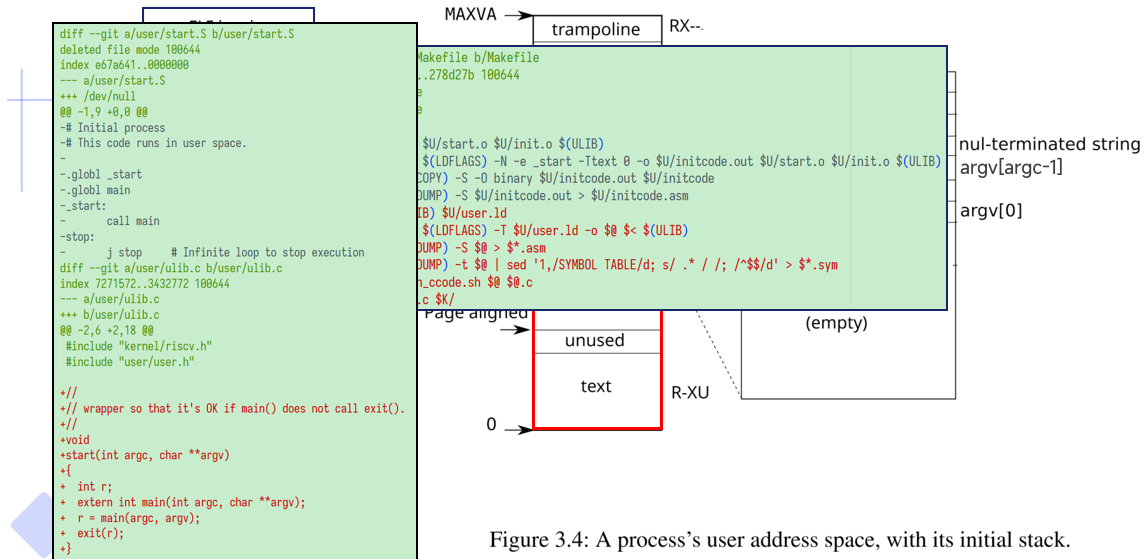


Figure 3.4: A process's user address space, with its initial stack.

完善的用户程序加载

```
diff --git a/user/start.S b/user/start.S
deleted file mode 100644
index e67a641..0000000
--- a/user/start.S
+++ /dev/null
@@ -1,9 +0,0 @@
-# Initial process
-# This code runs in user space.
-
-.globl _start
-.globl main
-_start:
-    call main
-
-stop:
-    j stop    # Infinite loop to stop execution
diff --git a/user/ulib.c b/user/ulib.c
index 7271572..3432772 100644
--- a/user/ulib.c
+++ b/user/ulib.c
@@ -2,6 +2,18 @@
#include "kernel/riscv.h"
#include "user/user.h"

//
// wrapper so that it's OK if main() does not call exit().
//
void
start(int argc, char **argv)
{
    int r;
    extern int main(int argc, char **argv);
    r = main(argc, argv);
    exit(r);
}
```

MAXVA →

trampoline

RX--.

```
Makefile b/Makefile
..278d27b 100644
```

```
$ riscv64-unknown-elf-readelf -h user/_init
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                           ELF64
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                        0
  Type:                               EXEC (Executable file)
  Machine:                            RISC-V
  Version:                            0x1
  Entry point address:                0x152
  Start of program headers:          64 (bytes into file)
  Start of section headers:         26728 (bytes into file)
  Flags:                              0x5, RVC, double-float ABI
  Size of this header:                64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:          4
  Size of section headers:           64 (bytes)
  Number of section headers:          18
  Section header string table index: 17
```

```
$ riscv64-unknown-elf-nm user/_init | grep start
0000000000000152 T start
```

完善的用户程序加载

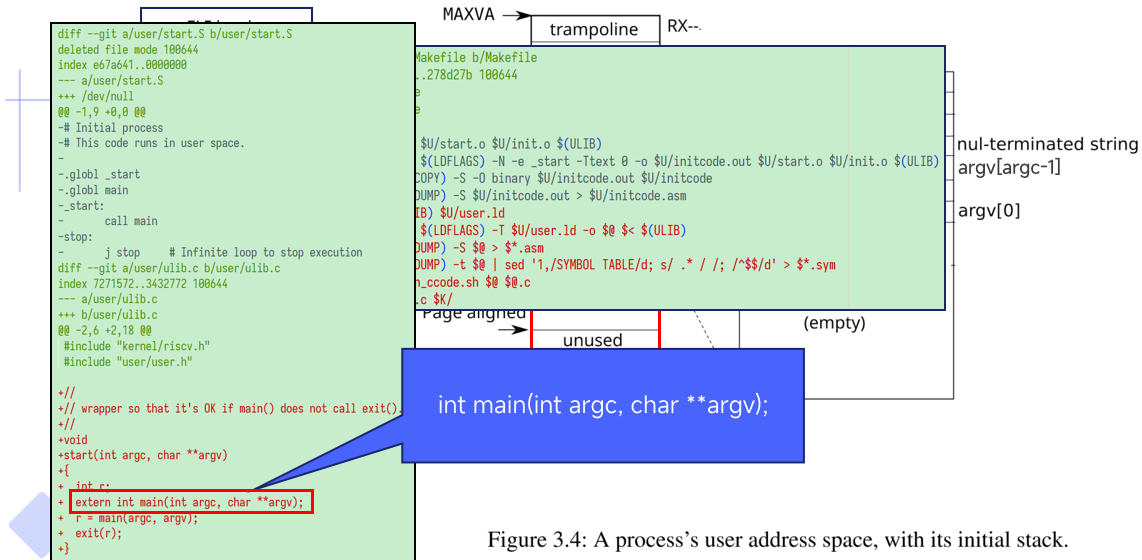
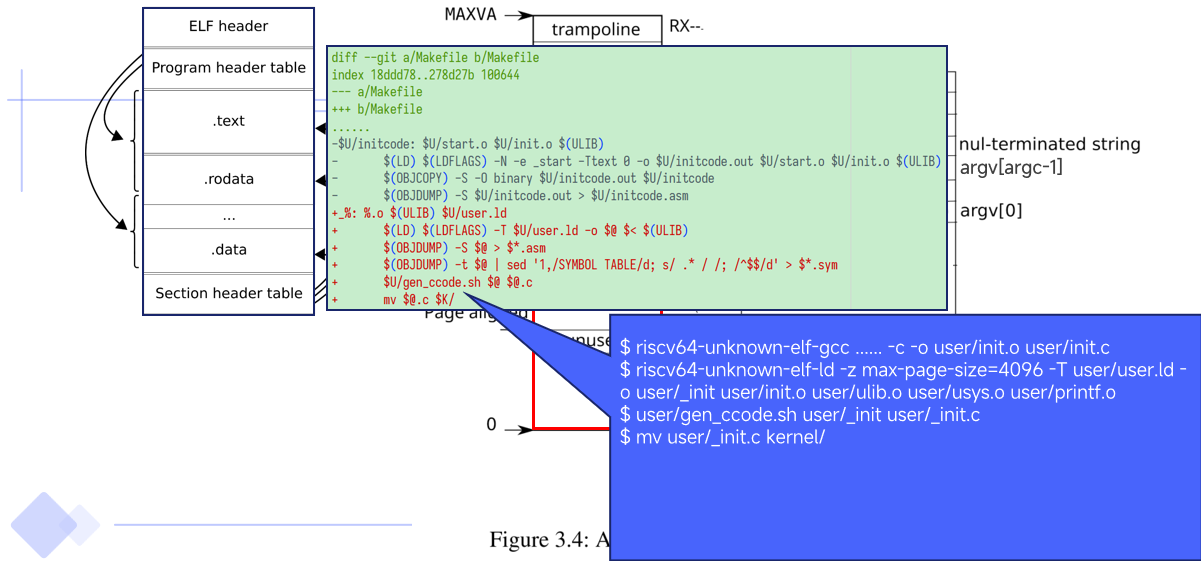
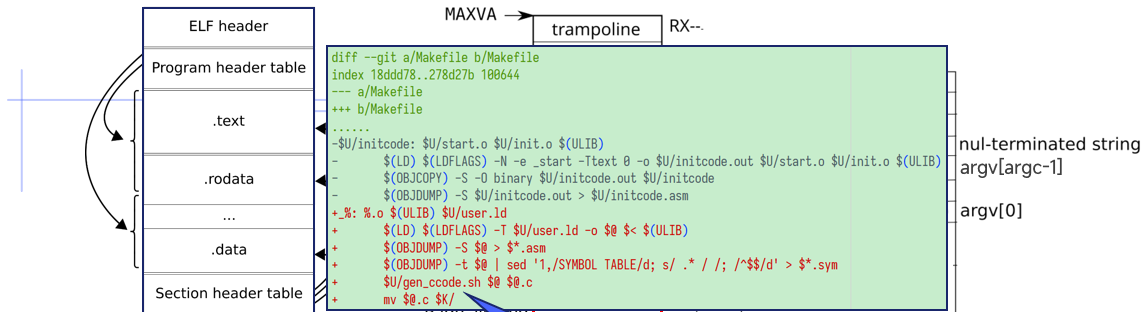


Figure 3.4: A process's user address space, with its initial stack.

完善的用户程序加载



完善的用户程序加载



kernel/_init.c

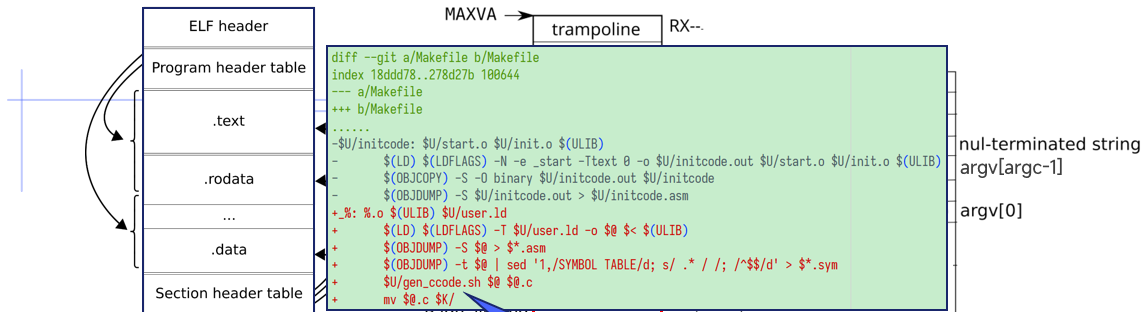
```
// Generated by gen_ccode.sh on Sun Jun 7 03:37:45 PM CST 2026 - do not edit
#include "types.h"

uchar program__init[] = {
0x7f, 0x45, 0x4c, 0x46, 0x02, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
// .....
};

int sizeof_program__init = sizeof(program__init);
```

```
$ riscv64-unknown-elf-gcc ..... -c -o user/init.o user/init.c
$ riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -
o user/_init user/init.o user/ulib.o user/usys.o user/printf.o
$ user/gen_ccode.sh user/_init user/_init.c
$ mv user/_init.c kernel/
```

完善的用户程序加载



kernel/_init.c

```
// Generated by gen_ccode.sh on Sun Jun  7 03:37:45 PM CST 2026 - do not edit
#include "types.h"

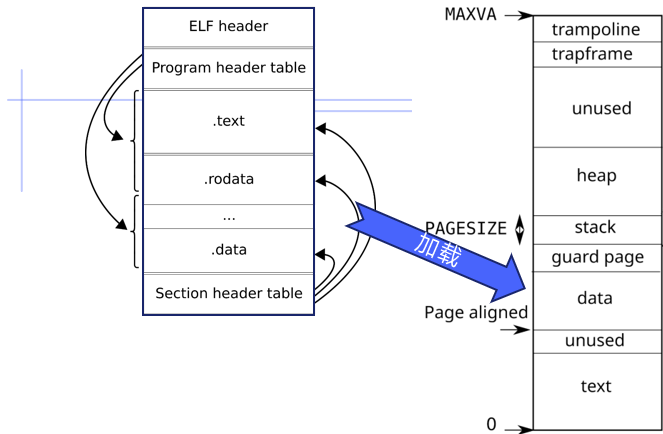
uchar program__init[] = {
0x7f, 0x45, 0x4c, 0x46, 0x02, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
// .....
};

int sizeof_program__init = sizeof(program__init);
```

```
$ riscv64-unknown-elf-gcc ..... -c -o user/init.o user/init.c
$ riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -
o user/_init user/init.o user/ulib.o user/usys.o user/printf.o
$ user/gen_ccode.sh user/_init user/_init.c
$ mv user/_init.c kernel/
```

```
.....
$ riscv64-unknown-elf-gcc ..... -c -o kernel/_init.o kernel/_init.c
$ riscv64-unknown-elf-ld -z max-page-size=4096 -T
kernel/kernel.ld -o kernel/kernel ..... kernel/_init.o
```

kexec 函数实现 (一)

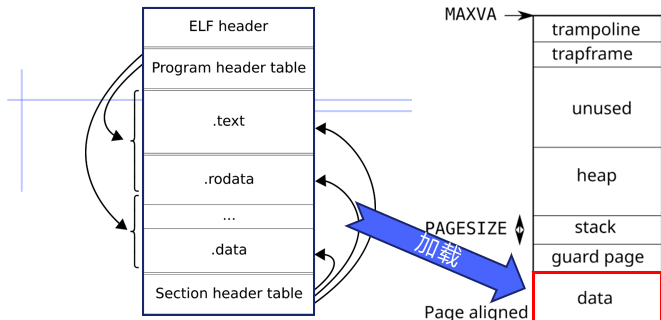


```
extern uchar program__init[];  
extern int sizeof_program__init;
```

```
int kexec(char *path, char **argv)  
{  
    // .....  
    struct elfhdr elf;  
    struct proc *p = myproc();  
    uchar *program = 0;  
    if (strncmp(path, "init", 5) == 0) {  
        program = program__init;  
    } else  
        goto bad;  
    // Read the ELF header.  
    memmove(&elf, program, sizeof(elf));  
    // Is this really an ELF file?  
    if(elf.magic != ELF_MAGIC)  
        goto bad;  
    // 未完待续 .....
```

Figure 3.4: A process

kexec 函数实现 (二.0)



```
int kexec(char *path, char **argv)
{
    // .....
    int i, off;
    uint64 sz = 0;
    struct elfhdr elf;
    struct proghdr ph;
    pagetable_t pagetable = 0, oldpagetable;
    struct proc *p = myproc();
    // ... 接 (一)
    if((pagetable = proc_pagetable(p)) == 0)
        goto bad;
    // Load program into memory.
    for(i=0, off=elf.phoff; i<elf.phnum; i++, off+=sizeof(ph)){
        memmove(&ph, program + off, sizeof(ph));
        if(ph.type != ELF_PROG_LOAD)
            continue;
        // .....
        uint64 sz1;
        if((sz1 = uvmmalloc(pagetable,
                             sz, ph.vaddr + ph.memsz,
                             flags2perm(ph.flags))) == 0)
            goto bad;
        sz = sz1;
        if(loadseg(pagetable, ph.vaddr, program,
                   ph.off, ph.filesz) < 0)
            goto bad;
    }
    // 未完待续
}
```

Figure 3.4: A process

```
$ riscv64-unknown-elf-readelf -lW user/_init
```

Elf file type is EXEC (Executable file)

Entry point 0x152

There are 4 program headers, starting at offset 64

Program Headers:

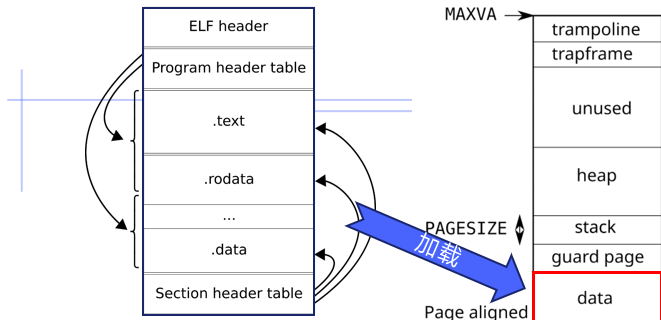
Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
RISCV_ATTRIBUTES	0x005edd	0x0000000000000000	0x0000000000000000	0x000074	0x000000	R	0x1
LOAD	0x001000	0x0000000000000000	0x0000000000000000	0x0000f9	0x000064	R E	0x1000
LOAD	0x002000	0x0000000000001000	0x0000000000001000	0x000000	0x000064	RW	0x1000
GNU_STACK	0x000000	0x0000000000000000	0x0000000000000000	0x000000	0x000000	RW	0x10

Section to Segment mapping:

Segment Sections...

Segment	Sections...
00	.riscv.attributes
01	.text .rodata
02	.data .bss
03	

kexec 函数实现 (二.1)



```
int kexec(char *path, char **argv)
{
    // .....
    int i, off;
    uint64 sz = 0;
    struct elfhdr elf;
    struct proghdr ph;
    pagetable_t pagetable = 0, oldpagetable;
    struct proc *p = myproc();
    // ... 接 (一)
    if((pagetable = proc_pagetable(p)) == 0)
        goto bad;
    // Load program into memory.
    for(i=0, off=elf.phoff; i<elf.phnum; i++, off+=sizeof(ph)){
        memmove(&ph, program + off, sizeof(ph));
        if(ph.type != ELF_PROG_LOAD)
            continue;
        // .....
        uint64 sz1;
        if((sz1 = uvmmalloc(pagetable,
                            sz, ph.vaddr + ph.memsz,
                            flags2perm(ph.flags))) == 0)
            goto bad;
        sz = sz1;
        if(loadseg(pagetable, ph.vaddr, program,
                    ph.off, ph.filesz) < 0)
            goto bad;
    }
    // 未完待续
}
```

Figure 3.4: A process

```
$ riscv64-unknown-elf-readelf -lW user/_init
```

```
Elf file type is EXEC (Executable file)
Entry point 0x152
There are 4 program headers, starting at offset 64
```

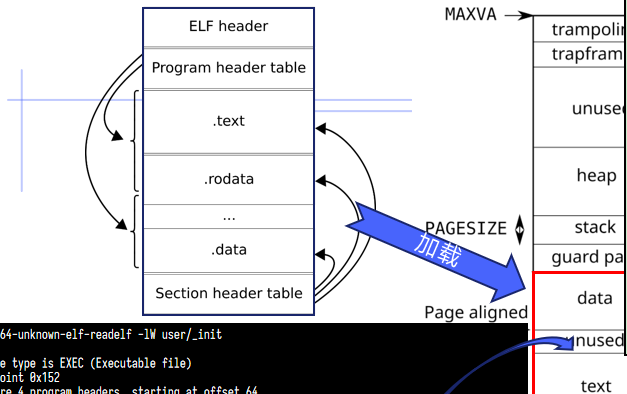
Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
RISCV_ATTRIBUTES	0x005edd	0x0000000000000000	0x0000000000000000	0x000074	0x000000	R	0x1
LOAD	0x001000	0x0000000000000000	0x0000000000000000	0x0000f9	0x000064	R E	0x1000
LOAD	0x002000	0x0000000000001000	0x0000000000001000	0x000000	0x000064	RW	0x1000
GNU_STACK	0x000000	0x0000000000000000	0x0000000000000000	0x000000	0x000000	RW	0x10

Section to Segment mapping:

Segment	Sections...
00	.riscv.attributes
01	.text .rodata
02	.data .bss
03	

kexec 函数实现 (二.2)



```
uint64_t uvmalloc(pagetable_t pagetable, uint64_t oldsz, uint64_t newsz, int xperm)
{
    char *mem;
    uint64_t a;
    if(newsz < oldsz)
        return oldsz;
    oldsz = PGROUNDUP(oldsz);
    for(a = oldsz; a < newsz; a += PGSIZE){
        mem = kalloc();
        if(mem == 0){
            uvmdealloc(pagetable, a, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        if(mappages(pagetable, a, PGSIZE, (uint64_t)mem, PTE_R|PTE_U|xperm) != 0){
            kfree(mem);
            uvmdealloc(pagetable, a, oldsz);
            return 0;
        }
    }
    return newsz;
}
```

```
$ riscv64-unknown-elf-readelf -lW user/_init
```

```
Elf file type is EXEC (Executable file)
Entry point 0x152
There are 4 program headers, starting at offset 64
```

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
RISCV_ATTRIBUTES	0x005edd	0x0000000000000000	0x0000000000000000	0x000074	0x000000	R	0x1
LOAD	0x001000	0x0000000000000000	0x0000000000000000	0x0000f9	0x000064	R E	0x1000
LOAD	0x002000	0x0000000000001000	0x0000000000001000	0x000000	0x000064	RW	0x1000
GNU_STACK	0x000000	0x0000000000000000	0x0000000000000000	0x000000	0x000000	RW	0x10

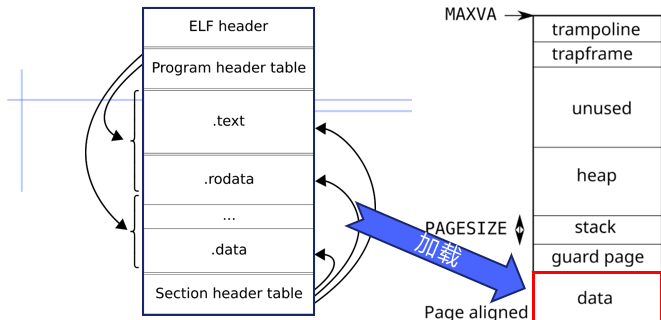
Section to Segment mapping:

Segment	Sections...
00	.riscv.attributes
01	.text .rodata
02	.data .bss
03	

```
if((sz1 = uvmalloc(pagetable,
                    sz, ph.vaddr + ph.memsz,
                    flags2perm(ph.flags))) == 0)
    goto bad;
sz = sz1;
if(loadseg(pagetable, ph.vaddr, program,
            ph.off, ph.filesz) < 0)
    goto bad;
// 未完待续
```

Figure 3.4: A process

kexec 函数实现 (二.3)



```
int kexec(char *path, char **argv)
{
    // .....
    int i, off;
    uint64 sz = 0;
    struct elfhdr elf;
    struct program_ph;

    static int
    loadseg(pagetable_t pagetable, uint64 va, uchar *ip, uint offset, uint sz)
    {
        uint i, n;
        uint64 pa;

        for(i = 0; i < sz; i += PGSIZE){
            pa = walkaddr(pagetable, va + i);
            if(pa == 0)
                panic("loadseg: address should exist");
            if(sz - i < PGSIZE)
                n = sz - i;
            else
                n = PGSIZE;
            memmove((uchar *)pa, ip + offset + i, n); // Copy from program to memory
        }

        return 0;
    }

    if(loadseg(pagetable, ph.vaddr, program,
               ph.off, ph.filesz) < 0)
        goto bad;

    // 未完待续
}
```

Figure 3.4: A process

```
$ riscv64-unknown-elf-readelf -lW user/_init
```

Elf file type is EXEC (Executable file)

Entry point 0x152

There are 4 program headers, starting at offset 64

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
RISCV_ATTRIBUTES	0x005edd	0x0000000000000000	0x0000000000000000	0x000074	0x000000	R	0x1
LOAD	0x001000	0x0000000000000000	0x0000000000000000	0x0006f9	0x0006f9	R E	0x1000
LOAD	0x002000	0x0000000000001000	0x0000000000001000	0x000000	0x000064	RW	0x1000
GNU_STACK	0x000000	0x0000000000000000	0x0000000000000000	0x000000	0x000000	RW	0x10

Section to Segment mapping:

Segment Sections...

Segment	Sections...
00	.riscv.attributes
01	.text .rodata
02	.data .bss
03	

kexec 函数实现 (三.0)

```
int kexec(char *path, char **argv)
{
    // ... 接 (二)
    uint64_t oldsz = p->sz;

    sz = PGROUNDUP(sz);
    uint64_t sz1;
    if((sz1 = uvmmalloc(pagetable, sz,
                        sz + (USERSTACK+1)*PGSIZE,
                        PTE_W)) == 0)
        goto bad;

    sz = sz1;
    uvmmclear(pagetable, sz-(USERSTACK+1)*PGSIZE);
    sp = sz;
    stackbase = sp - USERSTACK*PGSIZE;
    for(argc = 0; argv[argc]; argc++) {
        if(argc >= MAXARG)
            goto bad;
        sp -= strlen(argv[argc]) + 1;
        sp -= sp % 16; // riscv sp must be 16-byte aligned
        if(sp < stackbase)
            goto bad;
        if(copyout(pagetable, sp,
                  argv[argc], strlen(argv[argc]) + 1) < 0)
            goto bad;
        ustack[argc] = sp;
    }
    ustack[argc] = 0;
    sp -= (argc+1) * sizeof(uint64_t);
    sp -= sp % 16;
    if(sp < stackbase)
        goto bad;
    if(copyout(pagetable, sp,
              (char *)ustack, (argc+1)*sizeof(uint64_t)) < 0)
        goto bad;
    // 未完待续
}
```

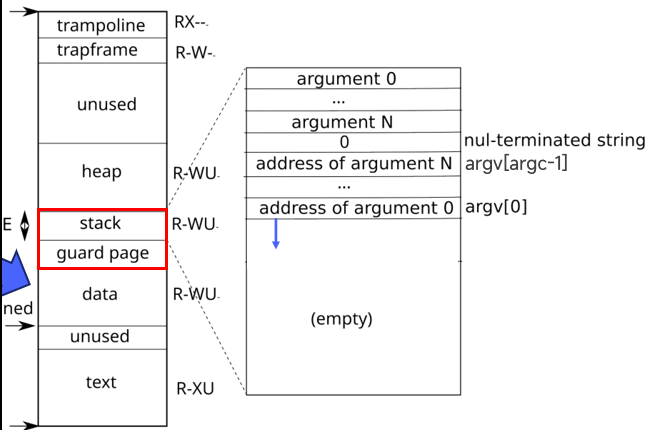


Figure 3.4: A process's user address space, with its initial stack.

kexec 函数实现 (三.1)

```
int kexec(char *path, char **argv)
{
    // ... 接 (二)
    uint64 oldsz = p->sz;

    sz = PGROUNDUP(sz);
    uint64 sz1;
    if((sz1 = uvmmalloc(pagetable, sz,
                        sz + (USERSTACK+1)*PGSIZE,
                        PTE_W)) == 0)
        goto bad;
    sz = sz1;
    uvmmclear(pagetable, sz-(USERSTACK+1)*PGSIZE);
    sp = sz;
    stackbase = sp - USERSTACK*PGSIZE;
    for(argc = 0; argv[argc]; argc++) {
        if(argc >= MAXARG)
            goto bad;
        sp -= strlen(argv[argc]) + 1;
        sp -= sp % 16; // riscv sp must be 16-byte aligned
        if(sp < stackbase)
            goto bad;
        if(copyout(pagetable, sp,
                  argv[argc], strlen(argv[argc]) + 1) < 0)
            goto bad;
        ustack[argc] = sp;
    }
    ustack[argc] = 0;
    sp -= (argc+1) * sizeof(uint64);
    sp -= sp % 16;
    if(sp < stackbase)
        goto bad;
    if(copyout(pagetable, sp,
              (char *)ustack, (argc+1)*sizeof(uint64)) < 0)
        goto bad;
    // 未完待续
}
```

```
// mark a PTE invalid for user access.
// used by exec for the user stack guard page.
void uvmmclear(pagetable_t pagetable, uint64 va)
{
    pte_t *pte;
    pte = walk(pagetable, va, 0);
    if(pte == 0)
        panic("uvmmclear");
    *pte &= ~PTE_U;
}
```

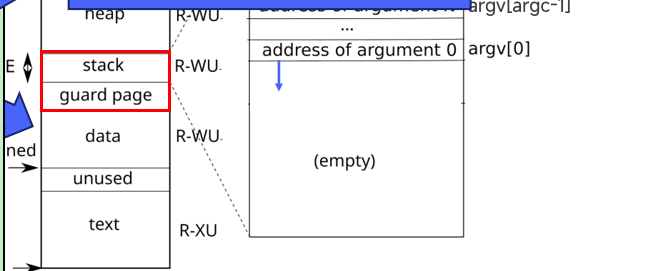


Figure 3.4: A process's user address space, with its initial stack.

kexec 函数实现 (三/2)

```
int kexec(char *path, char **argv)
{
    // ... 接 (二)
    uint64 oldsz = p->sz;

    sz = PGROUNDUP(sz);
    uint64 sz1;
    if((sz1 = uvmmalloc(pagetable, sz,
                        sz + (USERSTACK+1)*PGSIZE,
                        PTE_W)) == 0)
        goto bad;
    sz = sz1;
    uvmmclear(pagetable, sz-(USERSTACK+1)*PGSIZE);
    sp = sz;
    stackbase = sp - USERSTACK*PGSIZE;

    for(argc = 0; argv[argc]; argc++) {
        if(argc >= MAXARG)
            goto bad;
        sp -= strlen(argv[argc]) + 1;
        sp -= sp % 16; // riscv sp must be 16-byte aligned
        if(sp < stackbase)
            goto bad;
        if(copyout(pagetable, sp,
                  argv[argc], strlen(argv[argc]) + 1) < 0)
            goto bad;
        ustack[argc] = sp;
    }
    ustack[argc] = 0;
    sp -= (argc+1) * sizeof(uint64);
    sp -= sp % 16;
    if(sp < stackbase)
        goto bad;
    if(copyout(pagetable, sp,
              (char *)ustack, (argc+1)*sizeof(uint64)) < 0)
        goto bad;
    // 未完待续
}
```

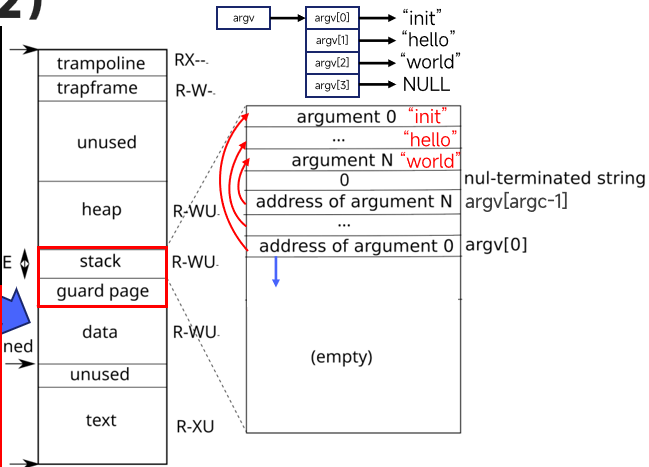


Figure 3.4: A process's user address space, with its initial stack.

kexec 函数实现 (四.0)

```
int kexec(char *path, char **argv)
{
    // .....
    p = myproc();
    uint64 oldsz = p->sz;
    // ... 接 (三)
    p->trapframe->a1 = sp;
    for(last=s=path; *s; s++)
        if(*s == '/')
            last = s+1;
    safestrcpy(p->name, last, sizeof(p->name));
    oldpagetable = p->pagetable;
    p->pagetable = pagetable;
    p->sz = sz;
    p->trapframe->epc = elf.entry;
    p->trapframe->sp = sp;
    proc_freepagetable(oldpagetable, oldsz);
    return argc;
bad:
    if(pagetable)
        proc_freepagetable(pagetable, sz);
    return -1;
}
```

int main(int argc, char **argv);

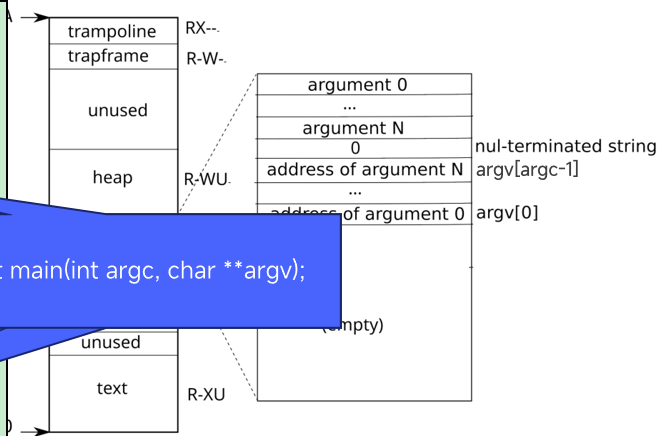


Figure 3.4: A process's user address space, with its initial stack.

kexec 函数实现 (四.0)

```
int kexec(char *path, char **argv)
{
    // .....
    p = myproc();
    uint64 oldsz = p->sz;
    // ... 接 (三)
    p->trapframe->a1 = sp;
    for(last=s=path; *s; s++)
        if(*s == '/')
            last = s+1;
    safestrcpy(p->name, last, sizeof(p->name));
    oldpagetable = p->pagetable;
    p->pagetable = pagetable;
    p->sz = sz;
    p->trapframe->epc = elf.entry;
    p->trapframe->sp = sp;
    proc_freepagetable(oldpagetable, oldsz);
    return argc;
bad:
    if(pagetable)
        proc_freepagetable(pagetable, sz);
    return -1;
}
```

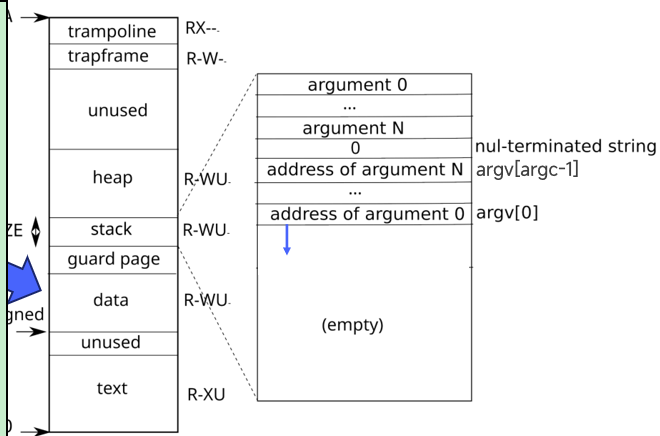
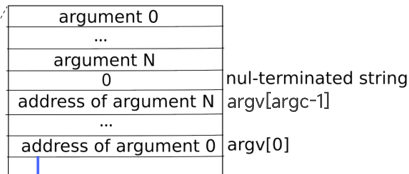
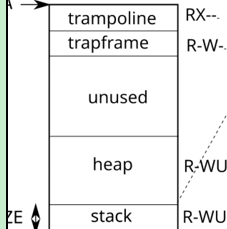


Figure 3.4: A process's user address space, with its initial stack.

kexec 函数实现 (四.1)

```
int kexec(char *path, char **argv)
{
    // .....
    p = myproc();
    uint64 oldsz = p->sz;
    // ... 接 (三)
    p->trapframe->a1 = sp;
    for(last=s=path; *s; s++)
        if(*s == '/')
            last = s+1;
    safestrcpy(p->name, last, sizeof(p->name));
    oldpagetable = p->pagetable;
    p->pagetable = pagetable;
    p->sz = sz;
    p->trapframe->epc = elf.entry;
    p->trapframe->sp = sp;
    proc_freepagetable(oldpagetable, oldsz);
    return argc;
bad:
    if(pagetable)
        proc_freepagetable(pagetable, sz);
    return -1;
}
```



kernel/trampoline.S

```
.globl uservec
uservec:
    ... # save context
    # call usertrap()
    jalr t0
.globl userret
userret:
    ... # restore context
    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    // .....
    if(r_scause() == 8){
        p->trapframe->epc =
        intr_on();
        syscall();
    }
    // .....
    return satp;
}
```

kernel/syscall.c

```
void syscall(void)
{
    int num;
    struct proc *p = myproc();
    num = p->trapframe->a7;
    p->trapframe->a0 = syscalls[num]();
}
```

执行程序 (exec)

- 系统调用 exec 的实现
- 核心函数 kexec 的实现
- **init 程序的加载**

init 程序的加载 - 回顾

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
```

```
static struct proc*
-allocproc(void (*start_routine)(void))
+allocproc(void)
{
    struct proc *p;
```

void scheduler(void)

```
{
    // .....
    switch(&c->context, &p->context);
    // .....
}
```

```
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

- p->start = start_routine;
-
    return p;
}
```

void forkret(void)

```
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();

    // Still holding p->lock from scheduler.
    release(&p->lock);

    if (first) {
        first = 0;
        // ensure other cores see first=0.
        __sync_synchronize();

        // We can invoke loadinit() now.
        // Put the return value (argc) of loadinit into a0.
        p->trapframe->a0 = loadinit(initcode, sizeof_initcode);
        if (p->trapframe->a0 == -1) {
            panic("loadinit");
        }
    }

    // return to user space, mimicing usertrap()'s return.
    prepare_return();
    uint64 satp = MAKE_SATP(p->pagetable);
    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
    ((void (*)(uint64))trampoline_userret)(satp);
}
```

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

init 程序的加载

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
```

```
static struct proc*
-allocproc(void (*start_routine)(void))
+allocproc(void)
{
    struct proc *p;
```

void scheduler(void)

```
{
    // .....
    switch(&c->context, &p->context);
    // .....
}
```

```
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

- p->start = start_routine;
-
    return p;
}
```

void forkret(void)

```
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();
```

```
// Still holding p->lock from scheduler.
release(&p->lock);
```

```
if (first) {
    first = 0;
    // ensure other cores see first=0.
    __sync_synchronize();
```

```
// We can invoke loadinit() now.
// Put the return value (argc) of loadinit into a0.
p->trapframe->a0 = loadinit(initcode, sizeof_initcode);
if (p->trapframe->a0 == -1) {
    panic("loadinit");
}
```

```
// return to user space, mimicing usertrap()'s return.
prepare_return();
uint64 satp = MAKE_SATP(p->pagetable);
uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
((void (*)(uint64))trampoline_userret)(satp);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fe06b8..b682be8 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -376,11 +341,11 @@ forkret(void)
    // ensure other cores see first=0.
    __sync_synchronize();
```

```
- // We can invoke loadinit() now.
- // Put the return value (argc) of loadinit into a0.
- p->trapframe->a0 = loadinit(initcode, sizeof_initcode);
+ // We can invoke kexec() now.
+ // Put the return value (argc) of kexec into a0.
+ p->trapframe->a0 = kexec("init", (char *)0){ "init", 0 });
+ if (p->trapframe->a0 == -1) {
-    panic("loadinit");
+    panic("kexec");
- }
+ }
```

sfence.vma zero, zero

jalr t0

userret:

sfence.vma zero, zero
csrw satp, a0
sfence.vma zero, zero

li a0, TRAMPOLINE
ld ra, 40(a0)
.....
ld t6, 280(a0)
ld a0, 112(a0)

sret

init 程序的加载

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
static struct proc*
-allocproc(void (*start_routine)(void))
+allocproc(void)
{
    struct proc *p;

    void scheduler(void)
    {
        // .....
        switch(&c->context, &p->context);
        // .....
    }
```

```
memset(&p->context, 0, sizeof(p->context));
- p->context.ra = (uint64)kerneltrapret;
+ p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

- p->start = start_routine;
-
    return p;
}
```

void forkret(void)

```
{
    extern char userret[];
    static int first = 1;
    struct proc *p = myproc();

    // Still holding p->lock from scheduler.
    release(&p->lock);

    if (first) {
        first = 0;
        // ensure other cores see first=0.
        __sync_synchronize();

        // We can invoke kexec() now.
        // Put the return value (argc) of kexec into a0.
        p->trapframe->a0 = kexec("init", (char *[]){ "init", 0 });
        if (p->trapframe->a0 == -1) {
            panic("kexec");
        }
    }

    // return to user space, mimicing usertrap()'s return.
    prepare_return();
    uint64 satp = MAKE_SATP(p->pagetable);
    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
    ((void (*)(uint64))trampoline_userret)(satp);
}
```

kernel/trampoline.S

```
uservec:
    csrw sscratch, a0
    li a0, TRAPFRAME
    sd ra, 40(a0)
    .....
    sd t6, 280(a0)
    csrr t0, sscratch
    sd t0, 112(a0)

    ld sp, 8(a0)
    ld tp, 32(a0)
    ld t0, 16(a0)
    ld t1, 0(a0)

    sfence.vma zero, zero
    csrw satp, t1
    sfence.vma zero, zero

    jalr t0

userret:
    sfence.vma zero, zero
    csrw satp, a0
    sfence.vma zero, zero

    li a0, TRAPFRAME
    ld ra, 40(a0)
    .....
    ld t6, 280(a0)
    ld a0, 112(a0)

    sret
```

init 程序的加载

kernel/proc.c

```
// Set up first user
void userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->state = RUNNABLE;
    release(&p->lock);
}
```

diff --git a/kernel/proc.c b/kernel/proc.c
index 9fb63dc..d2fe915 100644

```
--- a/kernel/proc.c  
+++ b/kernel/proc.c  
static struct proc*  
-allocproc(void (*start_routine)(void))  
+allocproc(void)  
{  
    struct proc *p;
```

void scheduler(void)

```
{  
    // .....  
    switch(&c->context, &p->context);  
    // .....  
}
```

```
memset(&p->context, 0, sizeof(p->context));  
- p->context.ra = (uint64)kerneltrapret;  
+ p->context.ra = (uint64)forkret;  
p->context.sp = p->kstack + PGSIZE;  
  
- p->start = start_routine;  
-  
    return p;  
}
```

void forkret(void)

```
{  
    extern char userret[];  
    static int first = 1;  
    struct proc *p = myproc();  
  
    // Still holding p->lock from scheduler.  
    release(&p->lock);  
  
    if (first) {  
        first = 0;  
        // ensure other cores see first=0.  
        __sync_synchronize();  
  
        // We can invoke kexec() now.  
        // Put the return value (argc) of kexec in  
        p->trapframe->a0 = kexec("init", (char *)  
        if (p->trapframe->a0 == -1) {  
            panic("exec");  
        }  
    }  
  
    // return to user space, mimicing usertrap()'s return.  
    prepare_return();  
    uint64 satp = MAKE_SATP(p->pagetable);  
    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);  
    ((void (*)(uint64))trampoline_userret)(satp);  
}
```

```
int kexec(char *path, char **argv)  
{  
    // .....  
    p = myproc();  
    uint64 oldsz = p->sz;  
    // ... 接 (三)  
    p->trapframe->a1 = sp;  
    for(last=s=path; *s; s++)  
        if(*s == '/')  
            last = s+1;  
    safestrcpy(p->name, last, sizeof(p->name));  
    oldpagetable = p->pagetable;  
    p->pagetable = pagetable;  
    p->sz = sz;  
    p->trapframe->epc = elf.entry;  
    p->trapframe->sp = sp;  
    proc_freepagetable(oldpagetable, oldsz);  
    return argc;  
bad:  
    if(pagetable)  
        proc_freepagetable(pagetable, sz);  
    return -1;  
}
```

csrw satp, a0
sfence.vma zero, zero

li a0, TRAPFRAME
ld ra, 40(a0)
.....
ld t6, 280(a0)
ld a0, 112(a0)

sret

init 程序的加载

kernel/proc.c

```
// Set up first
void userinit(void)
{
    struct proc *
    p = allocproc
    initproc = p;
    p->state = RUN
    release(&p->lock)
}
```

```
diff --git a/user/init.c b/user/init.c
index 638f662..f0a089f 100644
--- a/user/init.c
+++ b/user/init.c
```

```
@@ -24,10 +24,16 @@ getcmd(char *buf, int nbuf)
    return 0;
}
```

```
-void
-main(void)
```

```
+int
+main(int argc, char *argv[])
{
```

```
    static char buf[100];
    + int i;
```

```
    + printf("init: argc = %d\n", argc);
```

```
    + for(i = 0; i < argc; i++){
    +     printf("init: argv[%d] is: %s\n", i, argv[i]);
    + }
```

```
    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
```

```
@@ -39,4 +45,6 @@ main(void)
    cmd[strlen(cmd) - 1] = '\0';
    runcmd(cmd);
}
```

```
+ return 0;
```

```
}
```

forkr

ern c
tic i
uct p

Still
ease(

(first
first
/ ens

_sync

// We can invoke kexec() now.

// Put the return value (argc) of kexec into a0.

```
->trapframe->a0 = kexec("init", (char *[]){ "init", 0 });
```

```
f (p->trapframe->a0 == -1) {
```

```
    panic("exec");
```

return to user space, mimicing usertrap()'s return.

```
pare_return();
```

```
t64 satp = MAKE_SATP(p->pagetable);
```

```
t64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
```

```
oid (*)(uint64)trampoline_userret)(satp);
```

kernel/trampoline.S

userret:

csrw sscratch, a0

li a0, TRAMPOLINE

```
$ make qemu
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic
```

xv6 kernel is booting

hart 2 starting

hart 1 starting

init: argc = 1

init: argv[0] is: init

```
$
```

jair t0

userret:

sfence.vma zero, zero

csrw satp, a0

sfence.vma zero, zero

li a0, TRAMPOLINE

ld ra, 40(a0)

.....

ld t0, 280(a0)

ld a0, 112(a0)

sret

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc with dots at its ends is positioned around the top and right of the circles. In the top right corner, there is a solid blue rectangle.

04

创建进程 (fork)

进程操作模型 - fork

```
int pid, wpid, wstatus;

pid = fork(); // 父进程创建子进程
if (pid < 0) {
    // 创建进程失败
    return;
}

if (pid == 0) {
    // 子进程路径
    exec("xxxx", argv); // 执行程序 xxxx
    // 如果执行程序成功, exec 不会返回
    exit(1);
}

// 父进程路径
wpid = wait(&wstatus); // 父进程等待任一个子进程结束
if (wpid == pid) {
    // 子进程退出, wpid 是退出的子进程的 pid
    // wstatus 是子进程退出的状态值
} else {
    // wait 失败返回
}
```

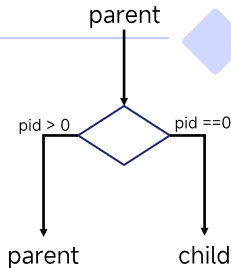
进程操作模型 - fork

```
int pid, wpid, wstatus;

pid = fork(); // 父进程创建子进程
if (pid < 0) {
    // 创建进程失败
    return;
}

if (pid == 0) {
    // 子进程路径
    exec("xxxx", argv); // 执行程序 xxxx
    // 如果执行程序成功, exec 不会返回
    exit(1);
}

// 父进程路径
wpid = wait(&wstatus); // 父进程等待任一个子进程结束
if (wpid == pid) {
    // 子进程退出, wpid 是退出的子进程的 pid
    // wstatus 是子进程退出的状态值
} else {
    // wait 失败返回
}
```



进程的父子关系

```
diff --git a/kernel/proc.h b/kernel/proc.h
index 08ad307..d0d8ea1 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -90,7 +90,7 @@ struct proc {
    void *chan;           // If non-zero, sleeping on chan
    int xstate;           // Exit status to be returned to parent's wait
    int pid;              // Process ID
-
+ struct proc *parent;    // Parent process
    // these are private to the process, so p->lock need not be held.
    uint64 kstack;        // Virtual address of kernel stack
    uint64 sz;            // Size of process memory (bytes)
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index b682be8..523cc01 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -156,6 +156,7 @@ freeproc(struct proc *p)
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
+   p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->xstate = 0;
@@ -220,6 +221,49 @@ userinit(void)
    release(&p->lock);
}

.....
@@ -460,7 +507,10 @@ procdump(void)
    state = states[p->state];
    else
        state = "???";
-   printf("%d %s %s", p->pid, state, p->name);
+   if (p->parent)
+       printf("%d %d %s %s", p->pid, p->parent->pid, state, p->name);
+   else
+       printf("%d x %s %s", p->pid, state, p->name);
    printf("\n");
}
```

系统调用 fork

fork(2) System Calls Manual fork(2)

NAME
fork - create a child process

LIBRARY
Standard C library (libc, -lc)

SYNOPSIS
`#include <unistd.h>`
`pid_t fork(void);`

man 2 fork

user/user.h

int fork(void);

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 3449c03..6d4ff15 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -80,6 +80,7 @@ argstr(int n, char *buf, int max)
 }

// Prototypes for the functions that handle system calls.
+extern uint64 sys_fork(void);
extern uint64 sys_exit(void);
extern uint64 sys_read(void);
extern uint64 sys_exec(void);
@@ -91,6 +92,7 @@ extern uint64 sys_write(void);
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
+ [SYS_fork] sys_fork,
 [SYS_exit] sys_exit,
 [SYS_read] sys_read,
 [SYS_exec] sys_exec,
```

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index 5246f73..b983f3e 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
@@ -21,6 +21,12 @@ sys_getpid(void)
 return myproc()->pid;
}

+uint64
+sys_fork(void)
+{
+ return kfork();
+}
+
```

系统调用 fork

```
int kfork(void)
{
    int pid;
    struct proc *np;
    struct proc *p = myproc();
    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }
    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;
    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);
    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;
    safestrcpy(np->name, p->name, sizeof(p->name));
    pid = np->pid;
    release(&np->lock);
    np->parent = p;
    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);
    return pid;
}
```

系统调用 fork

```
int kfork(void)
{
    int pid;
    struct proc *np;
    struct proc *p = myproc();
    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }
    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;
    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);
    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;
    safestrcpy(np->name, p->name, sizeof(p->name));
    pid = np->pid;
    release(&np->lock);
    np->parent = p;
    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);
    return pid;
}
```

创建新进程

```
static struct proc* allocproc(void)
{
    // .....

    // Set up new context to start executing at forkret,
    // which returns to user space.
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)forkret;
    p->context.sp = p->kstack + PGSIZE;

    return p;
}
```

系统调用 fork

```
int kfork(void)
{
    int pid;
    struct proc *np;
    struct proc *p = myproc();

    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }

    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;
    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);
    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;
    safestrcpy(np->name, p->name, sizeof(p->name));
    pid = np->pid;
    release(&np->lock);
    np->parent = p;
    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);
    return pid;
}
```

创建新进程

复制进程地址空间
(克隆)

```
int uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    char *mem;
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == 0)
            goto err;
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
            kfree(mem);
            goto err;
        }
    }
    return 0;
err:
    uvmunmap(new, 0, i / PGSIZE, 1);
    return -1;
}
```

系统调用 fork

```
int kfork(void)
{
    int pid;
    struct proc *np;
    struct proc *p = myproc();
    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }
    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;
    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);
    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;
    safestrcpy(np->name, p->name, sizeof(p->name));
    pid = np->pid;
    release(&np->lock);
    np->parent = p;
    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);
    return pid;
}
```

创建新进程

复制进程地址空间
(克隆)

复制进程的运行状态
并设置新进程返回用
户态时的返回值为 0

```
int uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    char *mem;
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == 0)
            goto err;
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
err:
            uvmunmap(new, 0, i / PGSIZE, 1);
            return -1;
        }
    }
}
```

系统调用 fork

```
int kfork(void)
```

```
{  
    int pid;  
    struct proc *np;  
    struct proc *p = myproc();  
    // Allocate process.  
    if((np = allocproc()) == 0){  
        return -1;  
    }
```

创建新进程

```
    // Copy user memory from parent to child.  
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){  
        freeproc(np);  
        release(&np->lock);  
        return -1;  
    }
```

复制进程地址空间
(克隆)

```
    np->sz = p->sz;  
    // copy saved user registers.  
    *(np->trapframe) = *(p->trapframe);  
    // Cause fork to return 0 in the child.  
    np->trapframe->a0 = 0;
```

建立父子关系

```
    safestrcpy(np->name, p->name, NAME_MAX, 1);  
    pid = np->pid;  
    release(&np->lock);  
    np->parent = p;  
    acquire(&np->lock);  
    np->state = RUNNABLE;  
    release(&np->lock);  
    return pid;  
}
```

```
int uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
```

```
{  
    pte_t *pte;  
    uint64 pa, i;  
    uint flags;  
    char *mem;  
    for(i = 0; i < sz; i += PGSIZE){  
        if((pte = walk(old, i, 0)) == 0)  
            panic("uvmcopy: pte should exist");  
        if((*pte & PTE_V) == 0)  
            panic("uvmcopy: page not present");  
        pa = PTE2PA(*pte);  
        flags = PTE_FLAGS(*pte);  
        if((mem = kalloc()) == 0)  
            goto err;  
        memmove(mem, (char*)pa, PGSIZE);  
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
```

复制进程的运行状态
并设置新进程返回用
户态时的返回值为 0

```
            err:  
            uvmunmap(new, 0, i / PGSIZE, 1);  
            return -1;  
        }  
    }
```

系统调用 fork

```
int kfork(void)
```

```
{  
    int pid;  
    struct proc *np;  
    struct proc *p = myproc();  
    // Allocate process.  
    if((np = allocproc()) == 0){  
        return -1;  
    }
```

创建新进程

```
    // Copy user memory from parent to child.  
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){  
        freeproc(np);  
        release(&np->lock);  
        return -1;  
    }
```

复制进程地址空间
(克隆)

```
    np->sz = p->sz;  
    // copy saved user registers.  
    *(np->trapframe) = *(p->trapframe);  
    // Cause fork to return 0 in the child.  
    np->trapframe->a0 = 0;
```

建立父子关系

```
    safestrcpy(np->name, p->name, NAME_MAX, 1);  
    pid = np->pid;  
    release(&np->lock);  
    np->parent = p;  
    acquire(&np->lock);  
    np->state = RUNNABLE;
```

子进程就绪

```
    release(&np->lock);  
    return pid;  
}
```

```
int uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
```

```
{  
    pte_t *pte;  
    uint64 pa, i;  
    uint flags;  
    char *mem;  
    for(i = 0; i < sz; i += PGSIZE){  
        if((pte = walk(old, i, 0)) == 0)  
            panic("uvmcopy: pte should exist");  
        if((*pte & PTE_V) == 0)  
            panic("uvmcopy: page not present");  
        pa = PTE2PA(*pte);  
        flags = PTE_FLAGS(*pte);  
        if((mem = kalloc()) == 0)  
            goto err;  
        memmove(mem, (char*)pa, PGSIZE);  
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
```

复制进程的运行状态
并设置新进程返回用
户态时的返回值为 0

```
            err:  
            uvmunmap(new, 0, i / PGSIZE, 1);  
            return -1;  
        }  
    }
```

系统调用 fork

```
int kfork(void)
```

```
{
    int pid;
    struct proc *np;
    struct proc *p = myproc();
    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }
    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;
    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);
    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;
    safestrcpy(np->name, p->name, NAME_MAX, 1);
    pid = np->pid;
    release(&np->lock);
    np->parent = p;
    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);
    return pid;
}
```

创建新进程

复制进程地址空间
(克隆)

建立父子关系

子进程就绪

对父进程返回子
进程的 pid

```
int uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
```

```
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    char *mem;
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == 0)
            goto err;
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
            err:
            uvmunmap(new, 0, i / PGSIZE, 1);
            return -1;
        }
    }
}
```

复制进程的运行状态
并设置新进程返回用
户态时的返回值为 0

系统调用 fork

```
int kfork(void)
```

```
{  
    int pid;  
    struct proc *np;  
    struct proc *p = myproc();
```

创建新进程

```
// Allocate process.  
if((np = allocproc()) == 0){  
    return -1;  
}
```

```
// Copy user memory from parent to child.
```

```
if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){  
    freeproc(np);  
    release(&np->lock);  
    return -1;  
}
```

复制进程地址空间
(克隆)

```
np->sz = p->sz;
```

```
// copy saved user registers.
```

```
*(np->trapframe) = *(p->trapframe);  
// Cause fork to return 0 in the child.
```

```
np->trapframe->a0 = 0;
```

```
safestrcpy(np->name, p->name, sizeof(np->name));
```

建立父子关系

```
pid = np->pid;
```

```
release(&np->lock);
```

```
np->parent = p;
```

子进程就绪

```
acquire(&np->lock);
```

```
np->state = RUNNABLE;
```

```
release(&np->lock);
```

```
return pid;
```

对父进程返回子
进程的 pid

```
int uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
```

```
{  
    pte_t *pte;  
    uint64 pa, i;  
    uint flags;  
    char *mem;  
    for(i = 0; i < sz; i += PGSIZE){  
        if((pte = walk(old, i, 0)) == 0)  
            panic("uvmcopy: pte should exist");  
        if((*pte & PTE_V) == 0)  
            panic("uvmcopy: page not present");  
        pa = PTE2PA(*pte);  
        flags = PTE_FLAGS(*pte);  
        if((mem = kalloc()) == 0)  
            goto err;  
        memmove(mem, (char*)pa, PGSIZE);  
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
```

复制进程的运行状态
并设置新进程返回用
户态时的返回值为 0

```
err:
```

```
    uvmunmap(new, 0, i / PGSIZE, 1);  
    return -1;  
}
```

parent

pid > 0

pid == 0

parent

child

系统调用 fork

user/init.c

```
char *argv[] = { "sh", 0 };
int main(void)
{
    int pid;
    for(;;){
        printf("init: starting sh\n");
        pid = fork();
        if(pid < 0){
            printf("init: fork failed\n");
            exit(1);
        }
        if(pid == 0){
            exec("sh", argv);
            printf("init: exec sh failed\n");
            exit(1);
        }
        // We have not supported wait in this version,
        // so we just wait indefinitely.
        while(1);
    }
}
```

user/sh.c

```
// Shell.
#include "kernel/types.h"
#include "user/user.h"

void runcmd(char *cmd)
{
    // .....
}

int getcmd(char *buf, int nbuf)
{
    // .....
}

void main(void)
{
    static char buf[100];

    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        char *cmd = buf;
        while (*cmd == ' ' || *cmd == '\t')
            cmd++;
        if (*cmd == '\n') // is a blank command
            continue;
        cmd[strlen(cmd) - 1] = '\0';
        runcmd(cmd);
    }
}
```

```
diff --git a/Makefile b/Makefile
index 278d27b..5b56909 100644
--- a/Makefile
+++ b/Makefile
@@ -23,6 +23,7 @@ OBJS = \
    $K/kernelvec.o \
    $K/plic.o \
    $K/_init.o \
+   $K/_sh.o \

# riscv64-unknown-elf- or riscv64-linux-gnu-
# perhaps in /opt/riscv/bin
@@ -80,7 +81,7 @@ endif

LDFLAGS = -z max-page-size=4096

-$K/kernel: $U/_init $(OBJS) $K/kernel.ld
+$K/kernel: $U/_init $U/_sh $(OBJS) $K/kernel.ld
    $(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
    $(OBJDUMP) -S $K/kernel > $K/kernel.asm
    $(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/ .
@@ -93,6 +94,8 @@ tags: $(OBJS)

$K/_init.o: $K/_init.c
    $(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
+$K/_sh.o: $K/_sh.c
+   $(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c

ULIB = $U/ulib.o $U/usys.o $U/printf.o

@@ -117,6 +120,7 @@ $U/usys.o : $U/usys.S

UPROGS=\
    $U/_init\
+   $U/_sh \
```

系统调用 fork

user/init.c

```
char *argv[] = { "sh", 0 };
int main(void)
{
    int pid;
    for(;;){
        printf("init: starting sh\n");
        pid = fork();
        if(pid < 0){
            printf("init: fork failed\n");
            exit(1);
        }
        if(pid == 0){
            exec("sh", argv);
            printf("init: exec sh failed\n");
            exit(1);
        }
        // We have not supported wait in this version,
        // so we just wait indefinitely.
        while(1);
    }
}
```

user/

```
// Shell
#include
#include

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
Running command: 'ls', unsupported!
$

void main(void)
{
    static char buf[100];

    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        char *cmd = buf;
        while (*cmd == ' ' || *cmd == '\t')
            cmd++;
        if (*cmd == '\n') // is a blank command
            continue;
        cmd[strlen(cmd) - 1] = '\0';
        runcmd(cmd);
    }
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic
```

```
-$K/kernel: $U/_init $(OBJJS) $K/kernel.ld
+$K/kernel: $U/_init $U/_sh $(OBJJS) $K/kernel.ld
$(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJJS)
$(OBJDUMP) -S $K/kernel > $K/kernel.asm
$(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/ .
@@ -93,6 +94,8 @@ tags: $(OBJJS)

$K/_init.o: $K/_init.c
$(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
+$K/_sh.o: $K/_sh.c
+ $(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c

ULIB = $U/ulib.o $U/usys.o $U/printf.o

@@ -117,6 +120,7 @@ $U/usys.o : $U/usys.S

UPROGS=\
$U/_init\
+ $U/_sh \
```

系统调用 fork

user/init.c

```
char *argv[] = { "sh", 0 };
int main(void)
{
    int pid;
    for(;;){
        printf("init: starting sh\n");
        pid = fork();
        if(pid < 0){
            printf("init: fork failed\n");
            exit(1);
        }
        if(pid == 0){
            exec("sh", argv);
            printf("init: exec sh failed\n");
            exit(1);
        }
        // We have not supported wait in this version,
        // so we just wait indefinitely.
        while(1);
    }
}
```

user/

```
// Shell
#include
#include

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
Running command: 'ls', unsupported!
$
1 x run    init
2 1 sleep  sh
```

Ctrl + P

```
void main(void)
{
    static char buf[100];

    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        char *cmd = buf;
        while (*cmd == ' ' || *cmd == '\t')
            cmd++;
        if (*cmd == '\n') // is a blank command
            continue;
        cmd[strlen(cmd) - 1] = '\0';
        runcmd(cmd);
    }
}
```

```
-$/kernel: $U/_init $(OBJJS) $K/kernel.ld
+$/kernel: $U/_init $U/_sh $(OBJJS) $K/kernel.ld
$(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJJS)
$(OBJDUMP) -S $K/kernel > $K/kernel.asm
$(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/ .
@@ -93,6 +94,8 @@ tags: $(OBJJS)

$/_init.o: $K/_init.c
$(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
+$/_sh.o: $K/_sh.c
+ $(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c

ULIB = $U/ulib.o $U/usys.o $U/printf.o

@@ -117,6 +120,7 @@ $U/usys.o : $U/usys.S

UPROGS=\
    $U/_init\
+    $U/_sh \
```



05

回收进程 (wait)

进程操作模型 - wait

```
int pid, wpid, wstatus;

pid = fork(); // 父进程创建子进程
if (pid < 0) {
    // 创建进程失败
    return;
}

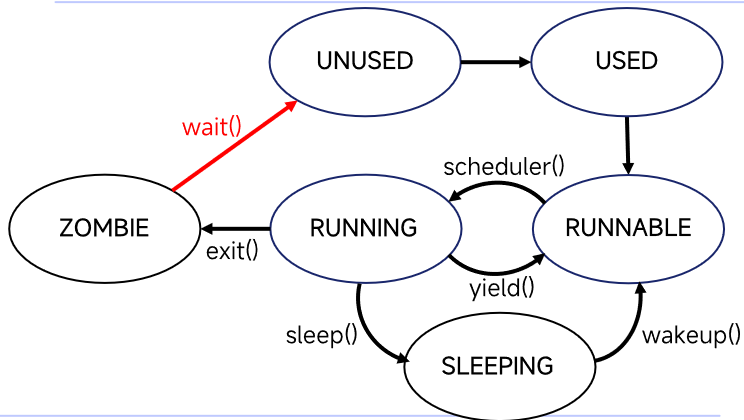
if (pid == 0) {
    // 子进程路径
    exec("xxxx", argv); // 执行程序 xxxx
    // 如果执行程序成功, exec 不会返回
    exit(1);
}

// 父进程路径
wpid = wait(&wstatus); // 父进程等待任一个子进程结束
if (wpid == pid) {
    // 子进程退出, wpid 是退出的子进程的 pid
    // wstatus 是子进程退出的状态值
} else {
    // wait 失败返回
}
```



回收进程 (wait)

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```



系统调用 wait

```
wait(2)                                     System Calls Manual                                     wait(2)

NAME
    wait, waitpid, waitid - wait for process to change state

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <sys/wait.h>

    pid_t wait(int * Nullable wstatus);
```

man 2 wait

user/user.h

int wait(int *wstatus);

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 6d4ff15..91bd019 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -82,6 +82,7 @@ argstr(int n, char *buf, int max)
 // Prototypes for the functions that handle system calls.
 extern uint64 sys_fork(void);
 extern uint64 sys_exit(void);
+extern uint64 sys_wait(void);
 extern uint64 sys_read(void);
 extern uint64 sys_exec(void);
 extern uint64 sys_getpid(void);
@@ -94,6 +95,7 @@ extern uint64 sys_write(void);
 static uint64 (*syscalls[])(void) = {
 [SYS_fork]    sys_fork,
 [SYS_exit]    sys_exit,
+[SYS_wait]    sys_wait,
 [SYS_read]    sys_read,
 [SYS_exec]    sys_exec,
 [SYS_getpid]  sys_getpid,
```

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index b983f3e..a5d4b86 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
@@ -27,6 +27,14 @@ sys_fork(void)
     return kfork();
 }

+uint64
+sys_wait(void)
+{
+    uint64 p;
+    argaddr(0, &p);
+    return kwait(p);
+}
```

系统调用 wait

```
int kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();
    acquire(&wait_lock);
    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                // make sure the child isn't still in exit() or swtch().
                acquire(&pp->lock);
                havekids = 1;
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp->xstate,
                                           sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                    freeproc(pp);
                    release(&pp->lock);
                    release(&wait_lock);
                    return pid;
                }
                release(&pp->lock);
            }
        }
        // No point waiting if we don't have any children.
        if(!havekids){
            release(&wait_lock);
            return -1;
        }
        // Wait for a child to exit.
        sleep(p, &wait_lock); //DOC: wait-sleep
    }
}
```

系统调用 wait

```
int kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();
    acquire(&wait_lock);
    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                // make sure the child isn't still in exit() or swtch().
                acquire(&pp->lock);
                havekids = 1;
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp->xstate,
                                           sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                    freeproc(pp);
                    release(&pp->lock);
                    release(&wait_lock);
                    return pid;
                }
                release(&pp->lock);
            }
        }
    }
    // No point waiting if we don't have any children.
    if(!havekids){
        release(&wait_lock);
        return -1;
    }
    // Wait for a child to exit.
    sleep(p, &wait_lock); //DOC: wait-sleep
}
}
```

系统调用 wait

```
int kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();
    acquire(&wait_lock);
    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                // make sure the child isn't still in exit() or swtch().
                acquire(&pp->lock);
                havekids = 1;
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp->xstate,
                                           sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                    freeproc(pp);
                    release(&pp->lock);
                    release(&wait_lock);
                    return pid;
                }
            }
            release(&pp->lock);
        }
        // No point waiting if we don't have any children.
        if(!havekids){
            release(&wait_lock);
            return -1;
        }
        // Wait for a child to exit.
        sleep(p, &wait_lock); //DOC: wait-sleep
    }
}
```

系统调用 wait

```
int kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();
    acquire(&wait_lock);
    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                // make sure the child isn't still in exit() or swtch().
                acquire(&pp->lock);
                havekids = 1;
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp->xstate,
                                           sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                    freeproc(pp);
                    release(&pp->lock);
                    release(&wait_lock);
                    return pid;
                }
            }
            release(&pp->lock);
        }
        // No point waiting if we don't have any children.
        if(!havekids){
            release(&wait_lock);
            return -1;
        }
        // Wait for a child to exit.
        sleep(p, &wait_lock); //DOC: wait-sleep
    }
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 523cc01..f49dcf2 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -275,16 +299,75 @@ kexit(int status)
     if(p == initproc)
         panic("init exiting");

+    + acquire(&wait_lock);
+    +
+    + // Give any children to init.
+    + reparent(p);
+    +
+    + // Parent might be sleeping in wait().
+    + wakeup(p->parent);
+    +
+    + acquire(&p->lock);
+
     p->xstate = status;
     p->state = ZOMBIE;

+    + release(&wait_lock);
+    +
+    // Jump into the scheduler, never to return.
+    sched();
+    panic("zombie exit");
+}
```

系统调用 wait

```
int kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();
    acquire(&wait_lock);
    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                // make sure the child isn't still in exit() or swtch().
                acquire(&pp->lock);
                havekids = 1;
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    if(addr != 0 && copyout(p->pagetable, addr, (char *)pp->xstate,
                                           sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                    freeproc(pp);
                    release(&pp->lock);
                    release(&wait_lock);
                    return pid;
                }
            }
            release(&pp->lock);
        }
        // No point waiting if we don't have any children.
        if(!havekids){
            release(&wait_lock);
            return -1;
        }
        // Wait for a child to exit.
        sleep(p, &wait_lock); //DOC: wait-sleep
    }
}
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 523cc01..f49dcf2 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -275,16 +299,75 @@ kexit(int status)
     if(p == initproc)
         panic("init exiting");
```

```
+ acquire(&wait_lock);
```

```
+ // Give any children to init.
+ reparent(p);
```

```
+ // Parent might be sleeping in wait.
```

```
+ wakeup(p->parent);
```

```
+ acquire(&p->lock);
```

```
p->xstate = status;
```

```
p->state = ZOMBIE;
```

```
+ release(&wait_lock);
```

```
+ // Jump into the scheduler, never to return.
```

```
sched();
```

```
panic("zombie exit");
```

```
}
```

```
void reparent(struct proc *p)
{
    struct proc *pp;
    for(pp = proc; pp < &proc[NPROC]; pp++){
        if(pp->parent == p){
            pp->parent = initproc;
            wakeup(initproc);
        }
    }
}
```

系统调用 wait

```
diff --git a/user/init.c b/user/init.c
index e7db18c..c699b2e 100644
--- a/user/init.c
+++ b/user/init.c
@@ -9,7 +9,7 @@ char *argv[] = { "sh", 0 };
int
main(void)
{
- int pid;
+ int pid, wpid;
  for(;;){
    printf("init: starting sh\n");
    pid = fork();
@@ -22,8 +22,21 @@ main(void)
    printf("init: exec sh failed\n");
    exit(1);
  }
- // We have not supported wait in this version,
- // so we just wait indefinitely.
- while(1);
+
+ for(;;){
+   // this call to wait() returns if the shell exits,
+   // or if a parentless process exits.
+   wpid = wait((int *) 0);
+   printf("init: wait returned successfully, process[%d] exited\n", wpid);
+   if(wpid == pid){
+     // the shell exited; restart it.
+     break;
+   } else if(wpid < 0){
+     printf("init: wait returned an error\n");
+     exit(1);
+   } else {
+     // it was a parentless process; do nothing.
+   }
+ }
}
```

系统调用 wait

```
diff --git a/user/init.c b/user/init.c
index e7db18c..c699b2e 100644
--- a/user/init.c
+++ b/user/init.c
@@ -9,7 +9,7 @@ char *argv[] = { "sh", 0 };
int
main(void)
{
- int pid;
+ int pid, wpid;
  for(;;){
    printf("init: starting sh\n");
    pid = fork();
@@ -22,8 +22,21 @@ main(void)
    printf("init: exec sh failed\n");
    exit(1);
  }
- // We have not supported wait in this version,
- // so we just wait indefinitely.
- while(1);
+
+ for(;;){
+   // this call to wait() returns if the shell exits,
+   // or if a parentless process exits.
+   wpid = wait((int *) 0);
+   printf("init: wait returned successfully, process[%d] exited\n", wpid);
+   if(wpid == pid){
+     // the shell exited; restart it.
+     break;
+   } else if(wpid < 0){
+     printf("init: wait returned an error\n");
+     exit(1);
+   } else {
+     // it was a parentless process; do nothing.
+   }
+ }
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic
```

xv6 kernel is booting

```
hart 1 starting
hart 2 starting
init: starting sh
```

```
$
1 x sleep  init
2 1 sleep  sh
```

Ctrl + P

系统调用 wait

```
diff --git a/user/init.c b/user/init.c
index e7db18c..c699b2e 100644
--- a/user/init.c
+++ b/user/init.c
@@ -9,7 +9,7 @@ char *argv[] = { "sh", 0 };
int
main(void)
{
- int pid;
+ int pid, wpid;
  for(;;){
    printf("init: starting sh\n");
    pid = fork();
@@ -22,8 +22,21 @@ main(void)
    printf("init: exec sh failed\n");
    exit(1);
  }
- // We have not supported wait in this version,
- // so we just wait indefinitely.
- while(1);
+
+ for(;;){
+   // this call to wait() returns if the shell exits,
+   // or if a parentless process exits.
+   wpid = wait((int *) 0);
+   printf("init: wait returned successfully, process[%d] exited\n", wpid);
+   if(wpid == pid){
+     // the shell exited; restart it.
+     break;
+   } else if(wpid < 0){
+     printf("init: wait returned an error\n");
+     exit(1);
+   } else {
+     // it was a parentless process; do nothing.
+   }
+ }
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic
```

xv6 kernel is booting

```
hart 1 starting
hart 2 starting
init: starting sh
```

```
$
1 x sleep  init
2 1 sleep  sh
```

```
$ exit
Exiting shell...
init: wait returned successfully, process[2] exited
init: starting sh
$
```

系统调用 wait

```
diff --git a/user/init.c b/user/init.c
index e7db18c..c699b2e 100644
--- a/user/init.c
+++ b/user/init.c
@@ -9,7 +9,7 @@ char *argv[] = { "sh", 0 };
int
main(void)
{
- int pid;
+ int pid, wpid;
  for(;;){
    printf("init: starting sh\n");
    pid = fork();
@@ -22,8 +22,21 @@ main(void)
    printf("init: exec sh failed\n");
    exit(1);
  }
- // We have not supported wait in this version,
- // so we just wait indefinitely.
- while(1);
+
+ for(;;){
+   // this call to wait() returns if the shell exits,
+   // or if a parentless process exits.
+   wpid = wait((int *) 0);
+   printf("init: wait returned successfully, process[%d] exited\n", wpid);
+   if(wpid == pid){
+     // the shell exited; restart it.
+     break;
+   } else if(wpid < 0){
+     printf("init: wait returned an error\n");
+     exit(1);
+   } else {
+     // it was a parentless process; do nothing.
+   }
+ }
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic
```

xv6 kernel is booting

```
hart 1 starting
hart 2 starting
init: starting sh
```

```
$
1 x sleep  init
2 1 sleep  sh
```

```
$ exit
Exiting shell...
init: wait returned successfully, process[2] exited
init: starting sh
$
1 x sleep  init
3 1 sleep  sh
```

Ctrl + P

系统调用 wait

```
diff --git a/user/init.c b/user/init.c
index e7db18c..c699b2e 100644
--- a/user/init.c
+++ b/user/init.c
@@ -9,7 +9,7 @@ char *argv[] = { "sh", 0 };
int
main(void)
{
- int pid;
+ int pid, wpid;
  for(;;){
    printf("init: starting sh\n");
    pid = fork();
@@ -22,8 +22,21 @@ main(void)
    printf("init: exec sh failed\n");
    exit(1);
  }
- // We have not supported wait in this version,
- // so we just wait indefinitely.
- while(1);
+
+ for(;;){
+   // this call to wait() returns if the shell exits,
+   // or if a parentless process exits.
+   wpid = wait((int *) 0);
+   printf("init: wait returned successfully, process[%d] exited\n", wpid);
+   if(wpid == pid){
+     // the shell exited; restart it.
+     break;
+   } else if(wpid < 0){
+     printf("init: wait returned an error\n");
+     exit(1);
+   } else {
+     // it was a parentless process; do nothing.
+   }
+ }
}
```

```
diff --git a/user/sh.c b/user/sh.c
index 638f662..cd685d0 100644
--- a/user/sh.c
+++ b/user/sh.c
@@ -3,12 +3,35 @@
#include "kernel/types.h"
#include "user/user.h"

+void cmd_zombie(void)
+{
+  printf("zombie: starting\n");
+  int pid = fork();
+  if(pid < 0) {
+    printf("zombie: fork failed\n");
+    exit(1);
+  }
+
+  // Parent exits first, leaving child as a zombie.
+  // The child will be reparented to init.
+  if (pid > 0) {
+    pause(50);
+    printf("zombie: parent[%d] exiting, child[%d] was adopted by init\n", getpid(), pid);
+  } else {
+    pause(100);
+    printf("zombie: child[%d] exiting\n", getpid());
+  }
+  exit(0);
+}

void
runcmd(char *cmd)
{
  if(strcmp(cmd, "exit") == 0) {
    printf("Exiting shell...\n");
    exit(0);
  } else if(strcmp(cmd, "zombie") == 0){
+    cmd_zombie();
  } else
    printf("Running command: '%s'\n", cmd);
}
```

系统调用 wait

```
$ make qemu  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3  
-nographic
```

xv6 kernel is booting

```
hart 1 starting  
hart 2 starting  
init: starting sh
```

\$

```
1 x sleep init
```

```
2 1 sleep sh
```

Ctrl + P

```
diff --git a/user/sh.c b/user/sh.c  
index 638f662..cd685d0 100644  
--- a/user/sh.c  
+++ b/user/sh.c  
@@ -3,12 +3,35 @@  
#include "kernel/types.h"  
#include "user/user.h"  
  
+void cmd_zombie(void)  
+{  
+    printf("zombie: starting\n");  
+    int pid = fork();  
+    if(pid < 0) {  
+        printf("zombie: fork failed\n");  
+        exit(1);  
+    }  
+  
+    // Parent exits first, leaving child as a zombie.  
+    // The child will be reparented to init.  
+    if (pid > 0) {  
+        pause(50);  
+        printf("zombie: parent[%d] exiting, child[%d] was adopted by init\n", getpid(), pid);  
+    } else {  
+        pause(100);  
+        printf("zombie: child[%d] exiting\n", getpid());  
+    }  
+    exit(0);  
+}  
  
void  
runcmd(char *cmd)  
{  
    if(strcmp(cmd, "exit") == 0) {  
        printf("Exiting shell...\n");  
        exit(0);  
    } else if(strcmp(cmd, "zombie") == 0){  
+        cmd_zombie();  
    } else  
        printf("Running command: '%s'\n", cmd);  
}
```

系统调用 wait

```
$ make qemu  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3  
-nographic
```

xv6 kernel is booting

```
hart 1 starting  
hart 2 starting  
init: starting sh
```

\$

```
1 x sleep init  
2 1 sleep sh
```

```
$ zombie  
zombie: starting
```

```
1 x sleep init  
2 1 sleep sh  
3 2 sleep sh
```

Ctrl + P

```
diff --git a/user/sh.c b/user/sh.c  
index 638f662..cd685d0 100644  
--- a/user/sh.c  
+++ b/user/sh.c  
@@ -3,12 +3,35 @@  
#include "kernel/types.h"  
#include "user/user.h"  
  
+void cmd_zombie(void)  
+{  
+    printf("zombie: starting\n");  
+    int pid = fork();  
+    if(pid < 0) {  
+        printf("zombie: fork failed\n");  
+        exit(1);  
+    }  
+  
+    // Parent exits first, leaving child as a zombie.  
+    // The child will be reparented to init.  
+    if (pid > 0) {  
+        pause(50);  
+        printf("zombie: parent[%d] exiting, child[%d] was adopted by init\n", getpid(), pid);  
+    } else {  
+        pause(100);  
+        printf("zombie: child[%d] exiting\n", getpid());  
+    }  
+    exit(0);  
+}  
  
void  
runcmd(char *cmd)  
{  
    if(strcmp(cmd, "exit") == 0) {  
        printf("Exiting shell...\n");  
        exit(0);  
+    } else if(strcmp(cmd, "zombie") == 0){  
+        cmd_zombie();  
+    } else  
        printf("Running command: '%s'\n", cmd);  
}
```

系统调用 wait

```
$ make qemu  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3  
-nographic
```

xv6 kernel is booting

```
hart 1 starting  
hart 2 starting  
init: starting sh
```

\$

```
1 x sleep init  
2 1 sleep sh
```

```
$ zombie  
zombie: starting
```

```
1 x sleep init  
2 1 sleep sh  
3 2 sleep sh
```

zombie: parent[2] exiting, child[3] was adopted by init

init: wait returned successfully, process[2] exited

init: starting sh

\$

```
diff --git a/user/sh.c b/user/sh.c  
index 638f662..cd685d0 100644  
--- a/user/sh.c  
+++ b/user/sh.c  
@@ -3,12 +3,35 @@  
#include "kernel/types.h"  
#include "user/user.h"  
  
+void cmd_zombie(void)  
+{  
+    printf("zombie: starting\n");  
+    int pid = fork();  
+    if(pid < 0) {  
+        printf("zombie: fork failed\n");  
+        exit(1);  
+    }  
+  
+    // Parent exits first, leaving child as a zombie.  
+    // The child will be reparented to init.  
+    if (pid > 0) {  
+        pause(50);  
+        printf("zombie: parent[%d] exiting, child[%d] was adopted by init\n", getpid(), pid);  
+    } else {  
+        pause(100);  
+        printf("zombie: child[%d] exiting\n", getpid());  
+    }  
+    exit(0);  
+}  
+  
void  
runcmd(char *cmd)  
{  
    if(strcmp(cmd, "exit") == 0) {  
        printf("Exiting shell...\n");  
        exit(0);  
+    } else if(strcmp(cmd, "zombie") == 0){  
+        cmd_zombie();  
+    } else  
        printf("Running command: '%s', unsupported!\n", cmd);  
}
```

系统调用 wait

```
$ make qemu  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3  
-nographic
```

xv6 kernel is booting

```
hart 1 starting  
hart 2 starting  
init: starting sh
```

\$

```
1 x sleep init  
2 1 sleep sh
```

```
$ zombie  
zombie: starting
```

```
1 x sleep init  
2 1 sleep sh  
3 2 sleep sh
```

```
zombie: parent[2] exiting, child[3] was adopted by init  
init: wait returned successfully, process[2] exited  
init: starting sh
```

\$

```
1 x sleep init  
4 1 sleep sh  
3 1 sleep sh
```

Ctrl + P

```
diff --git a/user/sh.c b/user/sh.c  
index 638f662..cd685d0 100644  
--- a/user/sh.c  
+++ b/user/sh.c  
@@ -3,12 +3,35 @@  
#include "kernel/types.h"  
#include "user/user.h"  
  
+void cmd_zombie(void)  
+{  
+    printf("zombie: starting\n");  
+    int pid = fork();  
+    if(pid < 0) {  
+        printf("zombie: fork failed\n");  
+        exit(1);  
+    }  
+  
+    // Parent exits first, leaving child as a zombie.  
+    // The child will be reparented to init.  
+    if (pid > 0) {  
+        pause(50);  
+        printf("zombie: parent[%d] exiting, child[%d] was adopted by init\n", getpid(), pid);  
+    } else {  
+        pause(100);  
+        printf("zombie: child[%d] exiting\n", getpid());  
+    }  
+    exit(0);  
+}  
+  
void  
runcmd(char *cmd)  
{  
    if(strcmp(cmd, "exit") == 0) {  
        printf("Exiting shell...\n");  
        exit(0);  
    } else if(strcmp(cmd, "zombie") == 0){  
+        cmd_zombie();  
    } else  
        printf("Running command: '%s'\n", cmd);  
}
```

系统调用 wait

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
-nographic
```

xv6 kernel is booting

```
hart 1 starting
hart 2 starting
init: starting sh
```

\$

```
1 x sleep init
2 1 sleep sh
```

```
$ zombie
zombie: starting
```

```
1 x sleep init
2 1 sleep sh
3 2 sleep sh
```

```
zombie: parent[2] exiting, child[3] was adopted by init
init: wait returned successfully, process[2] exited
init: starting sh
```

\$

```
1 x sleep init
4 1 sleep sh
3 1 sleep sh
```

```
zombie: child[3] exiting
init: wait returned successfully, process[3] exited
```

```
diff --git a/user/sh.c b/user/sh.c
index 638f662..cd685d0 100644
--- a/user/sh.c
+++ b/user/sh.c
@@ -3,12 +3,35 @@
#include "kernel/types.h"
#include "user/user.h"

+void cmd_zombie(void)
+{
+  printf("zombie: starting\n");
+  int pid = fork();
+  if(pid < 0) {
+    printf("zombie: fork failed\n");
+    exit(1);
+  }
+
+  // Parent exits first, leaving child as a zombie.
+  // The child will be reparented to init.
+  if (pid > 0) {
+    pause(50);
+    printf("zombie: parent[%d] exiting, child[%d] was adopted by init\n", getpid(), pid);
+  } else {
+    pause(100);
+    printf("zombie: child[%d] exiting\n", getpid());
+  }
+  exit(0);
+}

void
runcmd(char *cmd)
{
  if(strcmp(cmd, "exit") == 0) {
    printf("Exiting shell...\n");
    exit(0);
+  } else if(strcmp(cmd, "zombie") == 0){
+    cmd_zombie();
+  } else
    printf("Running command: '%s', unsupported!\n", cmd);
}
```

系统调用 wait

```
$ make qemu  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3  
-nographic
```

xv6 kernel is booting

```
hart 1 starting  
hart 2 starting  
init: starting sh
```

```
$  
1 x sleep init  
2 1 sleep sh
```

```
$ zombie  
zombie: starting
```

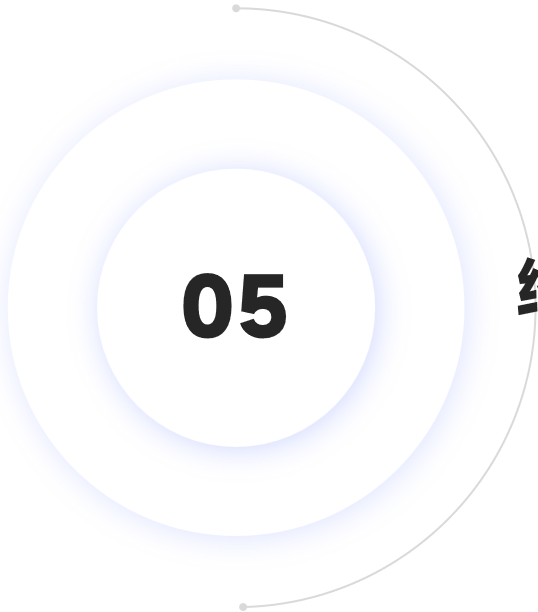
```
1 x sleep init  
2 1 sleep sh  
3 2 sleep sh  
zombie: parent[2] exiting, child[3] was adopted by init  
init: wait returned successfully, process[2] exited  
init: starting sh
```

```
$  
1 x sleep init  
4 1 sleep sh  
3 1 sleep sh  
zombie: child[3] exiting  
init: wait returned successfully, process[3] exited
```

```
1 x sleep init  
4 1 sleep sh
```

Ctrl + P

```
diff --git a/user/sh.c b/user/sh.c  
index 638f662..cd685d0 100644  
--- a/user/sh.c  
+++ b/user/sh.c  
@@ -3,12 +3,35 @@  
#include "kernel/types.h"  
#include "user/user.h"  
  
+void cmd_zombie(void)  
+{  
+    printf("zombie: starting\n");  
+    int pid = fork();  
+    if(pid < 0) {  
+        printf("zombie: fork failed\n");  
+        exit(1);  
+    }  
+  
+    // Parent exits first, leaving child as a zombie.  
+    // The child will be reparented to init.  
+    if (pid > 0) {  
+        pause(50);  
+        printf("zombie: parent[%d] exiting, child[%d] was adopted by init\n", getpid(), pid);  
+    } else {  
+        pause(100);  
+        printf("zombie: child[%d] exiting\n", getpid());  
+    }  
+    exit(0);  
+}  
  
void  
runcmd(char *cmd)  
{  
    if(strcmp(cmd, "exit") == 0) {  
        printf("Exiting shell...\n");  
        exit(0);  
    } else if(strcmp(cmd, "zombie") == 0){  
+        cmd_zombie();  
    } else  
        printf("Running command: '%s'\n", cmd);  
}
```



05

终止进程 (kill)



系统调用 kill

kill(2)

System Calls Manual

kill(2)

NAME

kill - send signal to a process

man 2 kill

LIBRARY

Standard C library (libc, -lc)

user/user.h

SYNOPSIS

#include <signal.h>

int kill(int pid);

int kill(pid_t pid, int sig);

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 91bd019..8955dca 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -84,6 +84,7 @@ extern uint64 sys_fork(void);
 extern uint64 sys_exit(void);
 extern uint64 sys_wait(void);
 extern uint64 sys_read(void);
+extern uint64 sys_kill(void);
 extern uint64 sys_exec(void);
 extern uint64 sys_getpid(void);
 extern uint64 sys_pause(void);
@@ -97,6 +98,7 @@ static uint64 (*syscalls[])(void) = {
 [SYS_exit]    sys_exit,
 [SYS_wait]    sys_wait,
 [SYS_read]    sys_read,
+[SYS_kill]    sys_kill,
 [SYS_exec]    sys_exec,
 [SYS_getpid]  sys_getpid,
 [SYS_pause]   sys_pause,
```

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index a5d4b86..ccc722f 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
+uint64
+sys_kill(void)
+{
+    int pid;
+
+    argint(0, &pid);
+    return kkill(pid);
+}
```

系统调用 kill

```
diff --git a/kernel/proc.h b/kernel/proc.h
index 656a006..3c33676 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -88,6 +88,7 @@ struct proc {
    // p->lock must be held when using these:
    enum procstate state;           // Process state
    void *chan;                     // If non-zero, sleeping on chan
+   int killed;                     // If non-zero, have been killed
    int xstate;                     // Exit status to be returned to parent's wait
    int pid;                         // Process ID
```

```
int kkill(int pid)
{
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
            }
            release(&p->lock);
            return 0;
        }
        release(&p->lock);
    }
    return -1;
}
```

系统调用 kill

```
diff --git a/kernel/proc.h b/kernel/proc.h
index 656a006..3c33676 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -88,6 +88,7 @@ struct proc {
    // p->lock must be held when using these:
    enum procstate state;           // Process state
    void *chan;                     // If non-zero, sleeping on chan
+   int killed;                     // If non-zero, have been killed
    int xstate;                     // Exit status to be returned to parent's wait
    int pid;                        // Process ID
```

```
int kkill(int pid)
{
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
            }
            release(&p->lock);
            return 0;
        }
        release(&p->lock);
    }
    return -1;
}
```

```
diff --git a/kernel/trap.c b/kernel/trap.c
index a7cf296..ab346cc 100644
--- a/kernel/trap.c
+++ b/kernel/trap.c
@@ -54,6 +54,9 @@ usertrap(void)
    if(r_scause() == 8){
        // system call

+       if(killed(p))
+           kexit(-1);
+
        // sepc points to the ecall instruction,
        // but we want to return to the next instruction.
        p->trapframe->epc += 4;
@@ -68,9 +71,12 @@ usertrap(void)
    } else {
        printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
        printf("            sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
-       panic("usertrap: should be killed but we have not support killing!");
+       setkilled(p);
    }

+   if(killed(p))
+       kexit(-1);
+
    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2)
        yield();
```

系统调用 kill

```
diff --git a/user/sh.c b/user/sh.c
index cd685d0..af776e0 100644
--- a/user/sh.c
+++ b/user/sh.c
@@ -32,6 +32,9 @@ runcmd(char *cmd)
    exit(0);
    } else if(strcmp(cmd, "zombie") == 0){
        cmd_zombie();
+   } else if(strcmp(cmd, "kill") == 0) {
+       printf("Oops, myself is killed...\n");
+       kill(getpid());
    } else
        printf("Running command: '%s', unsupported!\n", cmd);
}
```

系统调用 kill

```
diff --git a/user/sh.c b/user/sh.c
index cd685d0..af776e0 100644
--- a/user/sh.c
+++ b/user/sh.c
@@ -32,6 +32,9 @@ runcmd(char *cmd)
    exit(0);
    } else if(strcmp(cmd, "zombie") == 0){
        cmd_zombie();
+   } else if(strcmp(cmd, "kill") == 0) {
+       printf("Oops, myself is killed...\n");
+       kill(getpid());
    } else
        printf("Running command: '%s', unsupported\n", cmd);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
1 x sleep init
2 1 sleep sh
```

Ctrl + P

系统调用 kill

```
diff --git a/user/sh.c b/user/sh.c
index cd685d0..af776e0 100644
--- a/user/sh.c
+++ b/user/sh.c
@@ -32,6 +32,9 @@ runcmd(char *cmd)
    exit(0);
    } else if(strcmp(cmd, "zombie") == 0){
        cmd_zombie();
+   } else if(strcmp(cmd, "kill") == 0) {
+       printf("Oops, myself is killed...\n");
+       kill(getpid());
    } else
        printf("Running command: '%s', unsupported\n", cmd);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
1 x sleep  init
2 1 sleep  sh
Ctrl + P

$ kill
Oops, myself is killed...
init: starting sh
```

系统调用 kill

```
diff --git a/user/sh.c b/user/sh.c
index cd685d0..af776e0 100644
--- a/user/sh.c
+++ b/user/sh.c
@@ -32,6 +32,9 @@ runcmd(char *cmd)
    exit(0);
    } else if(strcmp(cmd, "zombie") == 0){
        cmd_zombie();
+   } else if(strcmp(cmd, "kill") == 0) {
+       printf("Oops, myself is killed...\n");
+       kill(getpid());
    } else
        printf("Running command: '%s', unsupported\n", cmd);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

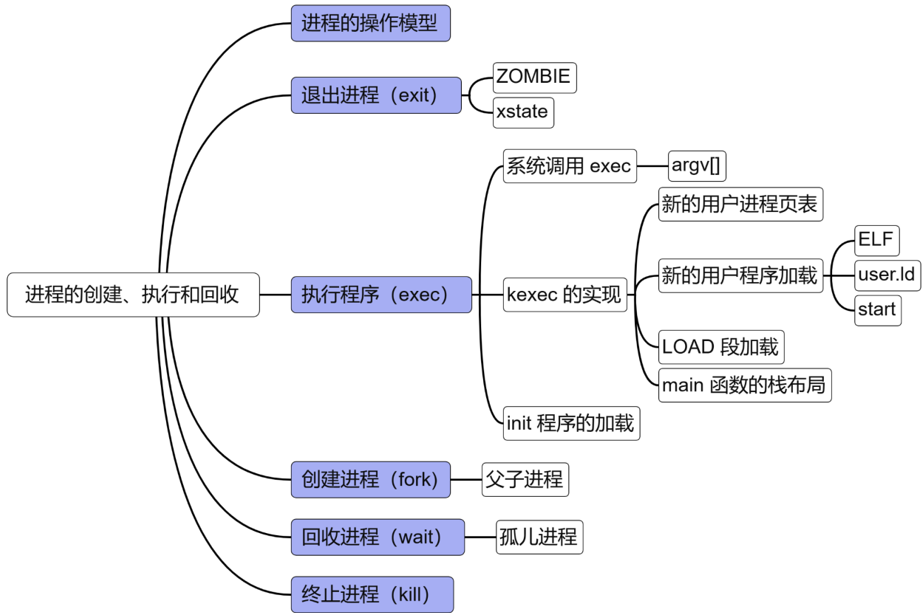
hart 1 starting
hart 2 starting
init: starting sh
$
1 x sleep init
2 1 sleep sh
Ctrl + P

$ kill
Oops, myself is killed...
init: starting sh
$
1 x sleep init
3 1 sleep sh
Ctrl + P

$
```



本章总结



谢谢

